

FINAL

LOW-LEVEL DESIGN DOCUMENT

Climate Intelligence Exchange

Grow+

End-to-End Architecture, Modules, Flows & Implementation Specification

Document Type	Final Low-Level Design Document
Project	Climate Intelligence Exchange (Grow+)
Version	V1.0
Total User Base	~2,000 NGO users (global)
Peak Concurrent Users	100–300
Daily Throughput	200–500 uploads · 1,000–3,000 queries
Classification	Confidential

Table of Contents

1. Introduction.....	7
1.1 What This Document Covers.....	7
1.2 The Three Core Flows.....	7
1.3 Operating Profile (Production).....	7
2. Architecture Overview.....	9
2.1 Architecture Principles.....	9
2.2 System-Wide Architecture Diagram.....	9
2.3 Why Two Services, Not Eight.....	10
3. Logical Modules — Complete Inventory.....	12
3.1 Module Inventory Table.....	12
3.2 Module Communication.....	12
4. Deployment Architecture.....	14
4.1 Production Deployment Grouping.....	14
4.2 Why ECS Fargate.....	14
4.3 Important Configuration Details.....	14
4.4 Deployment Pipeline.....	14
5. Flow 1 — Trust & Verification Flow.....	16
5.1 Modules Involved.....	16
5.2 Flow Diagram.....	16
5.3 Step-by-Step Specification.....	17
Step 1 — User Initiates Signup.....	17
Step 2 — Input Validation.....	18
Step 3 — Duplicate User Check.....	18
Step 4 — OTP Generation.....	18
Step 5 — OTP Storage in Postgres.....	18
Step 6 — Trigger Notification Service.....	19
Step 7 — Send OTP Email (Notification Service / Module 6).....	19
Step 8 — User Submits OTP.....	19
Steps 9–11 — OTP Validation.....	20
Step 12 — User Creation.....	20
Step 13 — User Profile Creation (User Service / Module 2).....	20
Steps 14–15 — Token Issuance.....	20
Step 16 — Response to Client.....	21
Step 17 — KYC Document Upload (Optional, Post-Verification).....	22
5.4 Token Refresh Flow.....	22
5.5 Logout.....	22
5.6 Security Considerations.....	22
5.7 Future Phase 2 Additions.....	23
6. Flow 2 — Data Ingestion Flow.....	24
6.1 Modules Involved.....	24
6.2 Flow Diagram.....	24
6.3 Step-by-Step Specification.....	25
Step 1 — Upload Request.....	25
Step 2 — Validation.....	25

Step 3 — Storage in AWS S3.....	26
Step 4 — Document Record Creation.....	26
Step 5 — Publish to SQS.....	26
Step 6 — SQS Consumer Picks Up Event.....	27
Step 7 — Document Parsing.....	28
Step 8 — OCR Fallback (Scanned Documents Only).....	28
Step 9 — Store Extracted Text.....	29
Step 10 — Text Chunking.....	29
Step 11 — Embedding Generation.....	29
Step 12 — LLM Climate Factor Extraction.....	30
Step 13 — Persist Structured Output.....	30
Step 14 — Refresh Search Index.....	31
Step 15 — Mark Document Completed.....	31
Step 16 — Acknowledge SQS Message.....	31
6.4 Status Polling (Frontend Integration).....	31
6.5 Status Lifecycle.....	32
6.6 Error Handling.....	32
6.7 Idempotency.....	32
6.8 SQS Configuration for This Flow.....	33
7. Flow 3 — Climate Twin Matching Flow.....	34
7.1 Modules Involved.....	34
7.2 The Five Steps.....	34
7.3 End-to-End Flow Diagram.....	34
7.4 Step A — User Input.....	36
A.1 Endpoint.....	36
A.2 Input Modes.....	36
A.3 Validation.....	36
A.4 query jobs Table.....	36
A.5 Publish to SQS.....	37
A.6 Response.....	37
7.5 Step B — Similarity Search (Engine Room).....	37
B.1 Trigger.....	37
B.2 Fetch Query Input.....	38
B.3 Embedding Generation.....	38
B.4 PGVector Similarity Search.....	38
B.5 Store Candidates.....	38
B.6 Update Status & Publish Next Event.....	39
7.6 Step C — Climate Twin Identification.....	39
C.1 Trigger.....	39
C.2 Fetch Climate Metadata for Candidates.....	39
C.3 Three-Layer Filtering.....	39
C.4 Select Top Twins.....	40
C.5 Store Climate Twins.....	40
C.6 Update Status & Publish Next Event.....	40
7.7 Step D — AI Synthesis (Knowledge Delivery).....	41
D.1 Trigger.....	41
D.2 Build LLM Context.....	41
D.3 Tiered LLM Strategy.....	41

D.4 Synthesis Prompt.....	41
D.5 LLM Call via LiteLLM.....	42
D.6 Rule-Based Fallback Triggers.....	42
D.7 Persist Result.....	42
D.8 Trigger Notification.....	43
D.9 Update Final Status.....	43
7.8 Step E — Result Delivery.....	43
E.1 Status Polling Endpoint.....	43
E.2 Result Endpoint.....	43
E.3 Authorisation.....	43
E.4 Response.....	44
E.5 Real-Time Push (SSE).....	44
7.9 Status Lifecycle (Full).....	44
7.10 SQS Configuration for This Flow.....	45
7.11 Cost Profile (Tiered LLM Strategy).....	45
8. Module Specifications.....	46
8.1 Module 1 — Auth Service.....	46
8.2 Module 2 — User Service.....	46
organisation_profiles schema.....	46
8.3 Module 3 — Document Service.....	47
8.4 Module 4 — Query Service.....	47
8.5 Module 5 — AI Intelligence Service.....	47
8.6 Module 6 — Notification Service.....	47
notifications schema.....	48
8.7 Module 7 — Admin Service.....	48
8.8 Module 8 — Search Service.....	48
global_search_index materialized view.....	49
Search query.....	50
Index refresh.....	50
9. Infrastructure & Deployment.....	51
9.1 Cloud Provider.....	51
9.2 Compute.....	51
9.3 Networking.....	51
9.4 Storage & Database.....	51
9.5 Messaging.....	51
9.6 CI/CD & IaC.....	52
9.6.1 Terraform Project Structure.....	52
9.6.2 Terraform State Management.....	53
9.6.3 Terraform Modules — Per-Resource Mapping.....	53
9.6.4 Deployment Pipeline for Terraform.....	54
9.6.5 Terraform Best Practices for This Project.....	54
9.7 Secrets Management.....	55
9.8 Estimated Monthly Cost.....	55
10. Observability & Operations.....	56
10.1 Final Pick.....	56
10.2 What Each Tool Catches.....	56
10.3 Alert Routing to Microsoft Teams.....	56
10.4 Day-to-Day Operations Workflow.....	56

10.5 Status Lifecycle Telemetry.....	57
11. Security & Compliance.....	58
11.1 Authentication & Authorization.....	58
11.2 Transport Security.....	58
11.3 Data Protection.....	58
11.4 Input Validation & Abuse Protection.....	58
11.5 Audit & Compliance.....	58
11.6 Secrets Management Summary.....	59
12. Critical Operational Considerations.....	60
12.1 Idempotency.....	60
12.2 Retry & Dead Letter Queue.....	60
12.3 Status Lifecycle (System-Wide).....	60
12.4 Error Recording.....	60
12.5 Rate Limiting.....	61
12.6 Data Lifecycle.....	61
12.7 Backup & Disaster Recovery.....	61
13. Production Hardening (v3.0 Updates).....	62
13.1 Failure Strategy — Retry, DLQ, and Failure State Tracking.....	62
13.1.1 Retry Policy.....	62
13.1.2 Schema Updates — retry count and error reason.....	62
13.1.3 Worker Implementation Pattern.....	63
13.1.4 Status Lifecycle (Updated).....	64
13.1.5 DLQ Operations.....	64
13.2 Observability — Structured Logs, Metrics, and Alerts.....	64
13.2.1 Structured JSON Logs (Mandatory).....	64
13.2.2 Metrics — Already Free via CloudWatch.....	65
13.2.3 Application-Level Metrics (CloudWatch Custom Metrics).....	65
13.2.4 Alert Routing.....	66
13.3 AI Layer Reliability — Validation, Confidence, and Human Review.....	66
13.3.1 Schema Validation (Hard Gate).....	66
13.3.2 Confidence Threshold and NEEDS REVIEW.....	66
13.3.3 Cross-Field Sanity Checks.....	67
13.3.4 Human-in-the-Loop Review (Module 7 — Admin Service).....	67
13.4 Idempotency — Preventing Duplicate Work.....	67
13.4.1 Idempotency Key on Every Event.....	68
13.4.2 Two-Layer Idempotency Check.....	68
13.4.3 Schema — processed events Table.....	68
13.5 File Storage — Pre-Signed URL Direct Upload to S3.....	69
13.5.1 Why This Matters.....	69
13.5.2 New Three-Call Upload Flow.....	69
13.5.3 Pre-Signed URL Generation (Go).....	70
13.5.4 S3 Bucket Policy — Hardening.....	70
13.5.5 CORS Configuration.....	70
13.6 Real-Time Result Delivery — SSE Primary, Polling Fallback.....	71
13.6.1 Why SSE, Not WebSockets.....	71
13.6.2 Polling as Fallback.....	71
13.7 Rate Limiting and Abuse Protection (Strengthened).....	71
13.7.1 Complete Rate Limit Matrix.....	71

13.7.2 Implementation.....	72
13.8 AI Output Versioning.....	72
13.8.1 Schema Updates.....	72
13.8.2 How It Is Used.....	73
13.9 Climate Data Source — Clarification and Phase 2 Plan.....	73
13.9.1 MVP Source — NGO-Authored Documents (Authoritative).....	73
13.9.2 Phase 2 — External Climate API Augmentation (Roadmap).....	73
13.9.3 Schema Provision for Phase 2.....	74
13.10 Multi-Tenancy — Organisation-Level Data Isolation.....	74
13.10.1 Tenant Boundary.....	74
13.10.2 Schema Enforcement.....	74
13.10.3 Row-Level Security via Application Layer.....	75
13.10.4 Cross-Org Visibility (The Climate Twin Exception).....	75
13.11 Summary — All v3.0 Changes At a Glance.....	75
14. Future Roadmap (Pie-in-Sky).....	77
14.1 Pie-in-Sky Features.....	77
14.2 Architectural Evolution.....	77
15. Final Word.....	78

1. Introduction

This Final Low-Level Design (LLD) document presents the complete, definitive technical architecture and component-level specifications for the Climate Intelligence Exchange platform — built for Grow+ to enable NGO's across the world to share climate knowledge, discover climate twins, and access funding pathways.

It incorporates every architectural decision the move to a modular monolith architecture, the consolidation of parsing into the Python AI service, all messaging, data, and infrastructure updates, and the selection of Terraform as the Infrastructure-as-Code tool.

1.1 What This Document Covers

- The complete platform architecture is organised into 8 logical modules across 2 deployable services
- Step-by-step low-level design for each of the 3 core platform flows
- Module-by-module specifications with technologies, responsibilities, API's, and database schemas
- Service communication maps, queue topology, and data flow diagrams
- Security, observability, and operational considerations
- Production deployment and scaling profile

1.2 The Three Core Flows

#	System Flow	Purpose	Covers
1	Trust & Verification Flow	To establish a secure, verified, and role-based identity system ensuring that all platform participants (NGOs, donors, admins) are authentic and authorized.	1. OTP-based signup (mobile/email verification) 2. Role assignment (NGO / Donor / Admin) 3. JWT-based authentication & session management 4. KYC / credential validation (if required) 5. Access control (RBAC)
2	Data Ingestion Flow	To collect, validate, process, and store NGO-submitted documents and transform them into structured and searchable data.	1. File upload (PDF / DOCX / TXT) 2. Validation (size, type, rate limiting) 3. Storage (S3 + metadata in

#	System Flow	Purpose	Covers
3	Climate Twin Matching Flow	To analyze ingested data using AI/LLMs and identify similar climate challenges (“Climate Twins”), enabling data-driven recommendations.	PostgreSQL 4. Parsing (Using Python Libraries) 5. Asynchronous processing via queue (SQS) 6. Extracted text persistence
			1. AI-based entity extraction (AQI, rainfall, temperature) 2. Text embedding generation 3. Storage in Vector DB (PGVector / Pinecone) 4. Semantic similarity search 5. AI-generated recommendations (RAG pipeline) 6. Async response delivery (polling / notifications)

1.3 Operating Profile (Production)

Parameter	Value
Total user base	~2,000 NGO users (global)
Peak concurrent users	100–300
Daily document uploads	200–500
Daily query submissions	1,000–3,000
Documents per query (RAG context)	3–5 climate twins
Average document size	Under 2 MB
Geographic concentration	Whole globe

Document Integrity Statement

This LLD is the authoritative engineering blueprint for the platform. Every technology choice, module boundary, API endpoint, queue topology, and data schema in this document has been finalised. Where an alternative approach exists, the reasoning for the chosen approach is preserved in the companion Final Tech Stack Selection document.

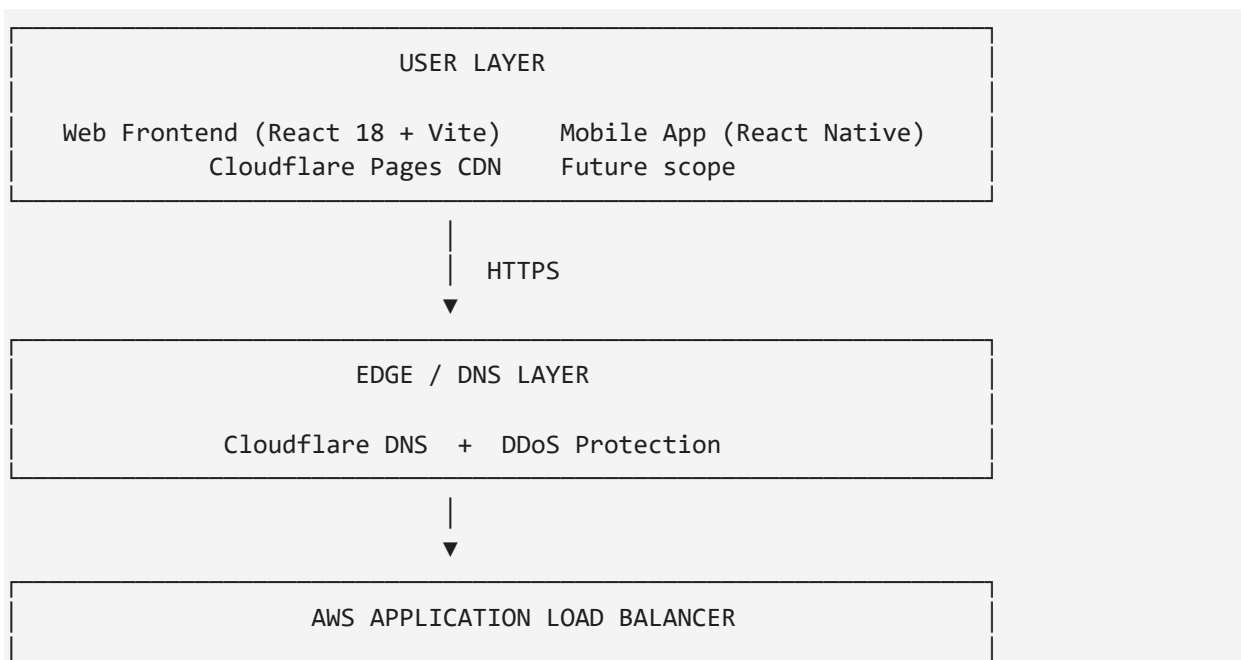
2. Architecture Overview

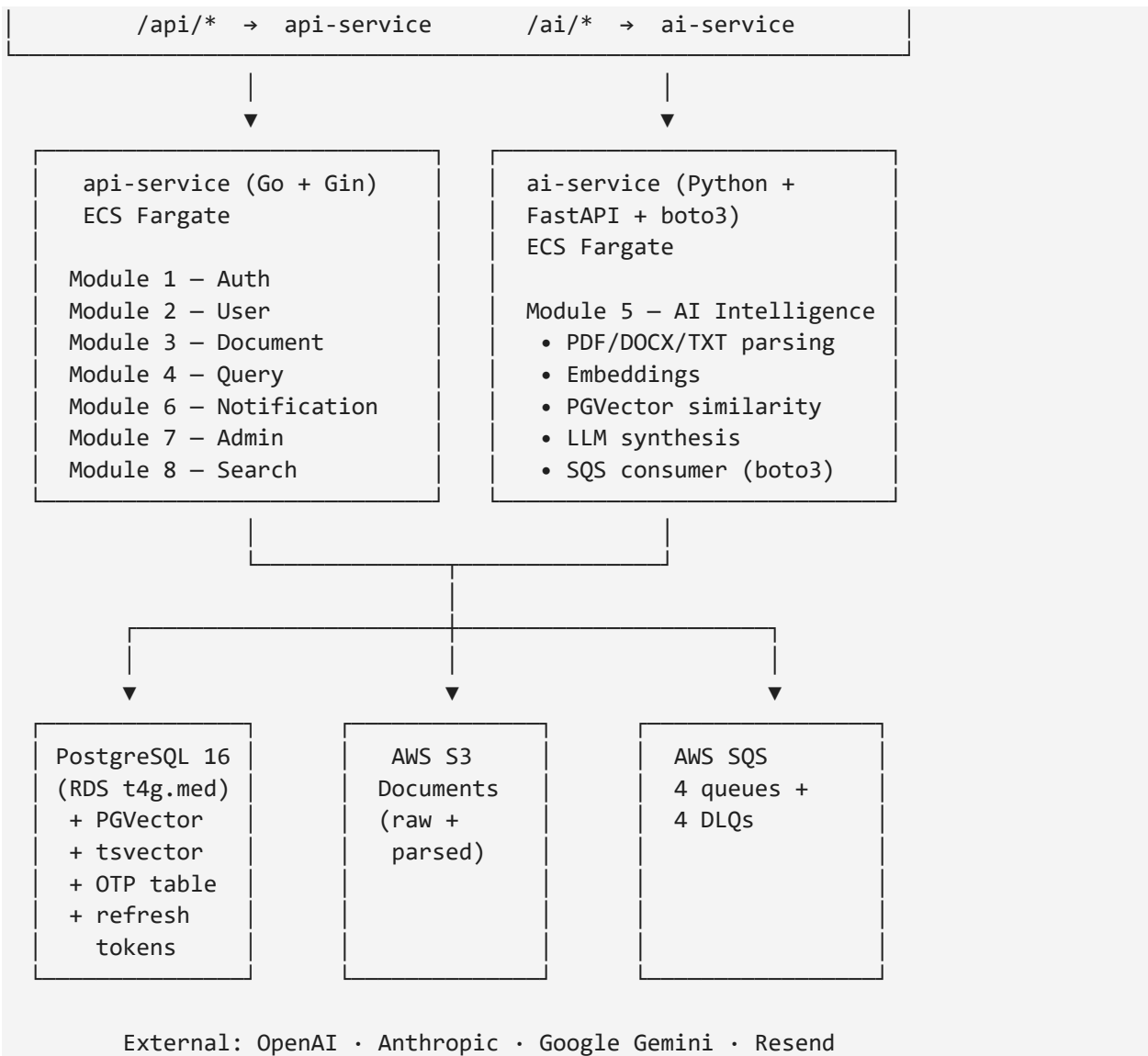
The platform is built as a modular monolith deployed as two services on AWS ECS Fargate. This is the simplest architecture that delivers production-grade reliability, supports the projected user load, and stays well within budget — without paying the operational cost of true microservices that the team does not need at this scale.

2.1 Architecture Principles

- Modular monolith — 8 logical modules deployed as 2 services. Modules talk via direct function calls inside the same Go binary, not over HTTP.
- Two languages — Go for the API and orchestration; Python for AI, parsing, embeddings, and LLM synthesis.
- Async by default — long-running operations (parsing, embedding, LLM synthesis) are queued via AWS SQS, never blocking API responses./
- Single AWS — one cloud provider, one IAM model, one bill, one networking model.
- Postgres-centric — PostgreSQL is the single source of truth for relational data, vector embeddings (PGVector), full-text search (tsvector), OTP storage, and refresh tokens.
- Cost-aware AI — tiered LLM strategy (Gemini Flash + Claude Haiku + Sonnet fallback) cuts AI costs by approximately 85%.

2.2 System-Wide Architecture Diagram





2.3 Why Two Services, Not Eight

The platform's eight logical modules deploy as two physical services. This is a deliberate decision based on the operating profile.

Question	Answer
Will it scale to 2,000 users?	Comfortably. A single Go binary on Fargate handles 10,000+ requests per second.
Will it scale to 10,000 users?	Yes. The same architecture, with horizontal scaling, handles this without changes.
Will it scale to 50,000 users?	Yes, with module extraction at that point — and only the specific modules under load need to be split.
Why not eight separate services	Eight services means eight CI pipelines, eight monitoring

Question	Answer
now?	setups, eight deployment targets, and zero added benefit at this scale. The team would pay full microservices cost for no reward.

Future Scaling Path

When a specific module shows measured scaling pain — for example the AI Intelligence service hitting LLM rate limits, or the Auth module being hit by sudden signup spikes — that module can be extracted into its own service in approximately 1 to 5 days.

3. Logical Modules — Complete Inventory

The platform is composed of 8 logical modules. Each owns a specific area of responsibility, exposes its own API surface, and may be extracted into its own service in the future. For the production deployment, modules 1, 2, 3, 4, 6, 7, and 8 ship inside the Go api-service binary; module 5 ships as the Python ai-service.

3.1 Module Inventory Table

#	Module	Tech Stack	Responsibility
1	Auth Service	Go + Gin + JWT + Argon2id + Resend	Signup, email OTP, login, refresh tokens (Postgres), RBAC, KYC workflow integration
2	User Service	Go + Gin	User profiles, organisation profiles, donor profiles, role separation
3	Document Service	Go + Gin + AWS S3 SDK	Upload validation, S3 storage, metadata, lifecycle. Parsing handled by ai-service.
4	Query Service	Go + Gin	Query submission, status, result retrieval (Steps A & E of Climate Twin Flow)
5	AI Intelligence Service	Python 3.12 + FastAPI + LiteLLM + PyMuPDF + python-docx + Tesseract + boto3	Document parsing, embedding generation, similarity search via PGVector, climate twin identification, LLM synthesis, direct SQS queue consumption
6	Notification Service	Go + Gin + Resend	Email OTP delivery, transactional email, in-app SSE push notifications
7	Admin Service	Go + Gin	KYC approval, NGO verification, content moderation, climate twin override (manual)
8	Search Service	Go + Gin + Postgres tsvector + PGVector	Global search across NGO profiles, documents, solutions, and climate twins via materialized view

3.2 Module Communication

Modules inside the api-service Go binary communicate via direct function calls — there is no HTTP between them. The only true microservice boundary is between api-service and ai-service, which communicate exclusively via AWS SQS queues.

Inside api-service (Go binary)

```

Auth — function call —> User
Auth — function call —> Notification
Doc  — function call —> Notification
Query — function call —> Search

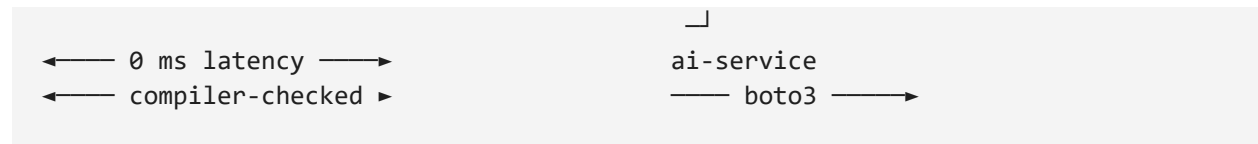
```

Across services

```

api-service
|
| AWS SQS queues
| (4 queues)

```



4. Deployment Architecture

The eight logical modules deploy as exactly two ECS Fargate services.

4.1 Production Deployment Grouping

Deployable Service	Tech	Contains Modules	Scaling Profile
api-service	Go binary on ECS Fargate	1 (Auth), 2 (User), 3 (Document), 4 (Query), 6 (Notification), 7 (Admin), 8 (Search)	0.5 vCPU / 1 GB RAM per task Minimum: 2 tasks (HA baseline) Maximum: 10 tasks Auto-scaling triggers: - Scale out when: - CPU > 70% for 2 consecutive minutes - OR Request latency > 300ms (p95) - Scale in when: - CPU < 30% for 5 minutes Scaling policy: — Target tracking scaling (CPU-based) — Cooldown: 120 seconds (scale out), 300 seconds (scale in) — Scale out when SQS queue depth > 100 messages — Scale to max capacity if queue > 500 messages
ai-service	Python on ECS Fargate	5 (AI Intelligence) — parsing, embedding, RAG, LLM	Compute: - 1 vCPU / 4 GB RAM per

Deployable Service	Tech	Contains Modules	Scaling Profile
		synthesis, SQS consumer (boto3)	task (baseline) Capacity: - Minimum: 1 task (cost-efficient baseline) - Maximum: 6 tasks (burst handling for ingestion spikes) Auto-scaling triggers: - Scale out when: - SQS queue depth > 50 messages - OR Average message processing time > 10 seconds - OR CPU utilization > 70% for 2 minutes - Scale aggressively when: - SQS queue depth > 200 messages → scale to max (6 tasks) Scale-in conditions: - Queue depth < 10 messages for 10 minutes - AND CPU < 30% Scaling policy: - Primary: Step scaling (based on SQS queue depth) - Secondary: Target tracking (CPU utilization) Cooldown: - Scale out: 60–120 seconds (fast reaction) - Scale in: 300–600 seconds (avoid thrashing)

4.2 Why ECS Fargate

- Serverless containers — no servers to patch, no clusters to manage.
- Pay per second of CPU and memory used, not for idle capacity.
- Native integration with ALB, ECR, IAM, Secrets Manager, Parameter Store.
- Approximately \$45 per month total for both services at this scale.

4.3 Important Configuration Details

Configuration	Value	Reason
ai-service stopTimeout	120 seconds	LLM calls can take 30–90 seconds. The default 30s would kill in-flight jobs.
ai-service RAM	4 GB minimum	Embedding models and LLM responses are memory-intensive.
ai-service health check grace period	60 seconds	Allows time for SQS consumer threads to start.
ECR lifecycle policy	Keep last 10 images	Prevents storage cost creep over time.
ALB target health interval	30 seconds	Standard balanced check frequency.

4.4 Deployment Pipeline

Two independent deployment pipelines, one per service:

Push to main branch (GitHub / GitLab)

- Pre-Build Checks (Common for both services)
 1. Run linting (golangci-lint / flake8)
 2. Run unit tests
 3. Run security scan (Snyk / Trivy)
 4. Validate dependencies
- deploy-api job
 5. Build Go binary
 6. Build Docker image
 7. Scan Docker image (Trivy)
 8. Push to AWS ECR
 9. Run DB migrations (if any)
 10. Deploy to ECS (rolling update)
 11. Wait for service stabilization
 12. Run smoke tests (API health + auth + ingestion)
 13. Monitor logs (CloudWatch alarms check)
- deploy-ai job
 5. Build Python Docker image
 6. Install dependencies (requirements.txt)

7. Scan Docker image (Trivy)
8. Push to AWS ECR
9. Deploy to ECS (rolling update)
10. Wait for stabilization
11. Run smoke tests (queue consumption + AI response)
12. Validate SQS processing health

If ai-service deployment fails, api-service is unaffected, and vice versa. Each deploys independently.

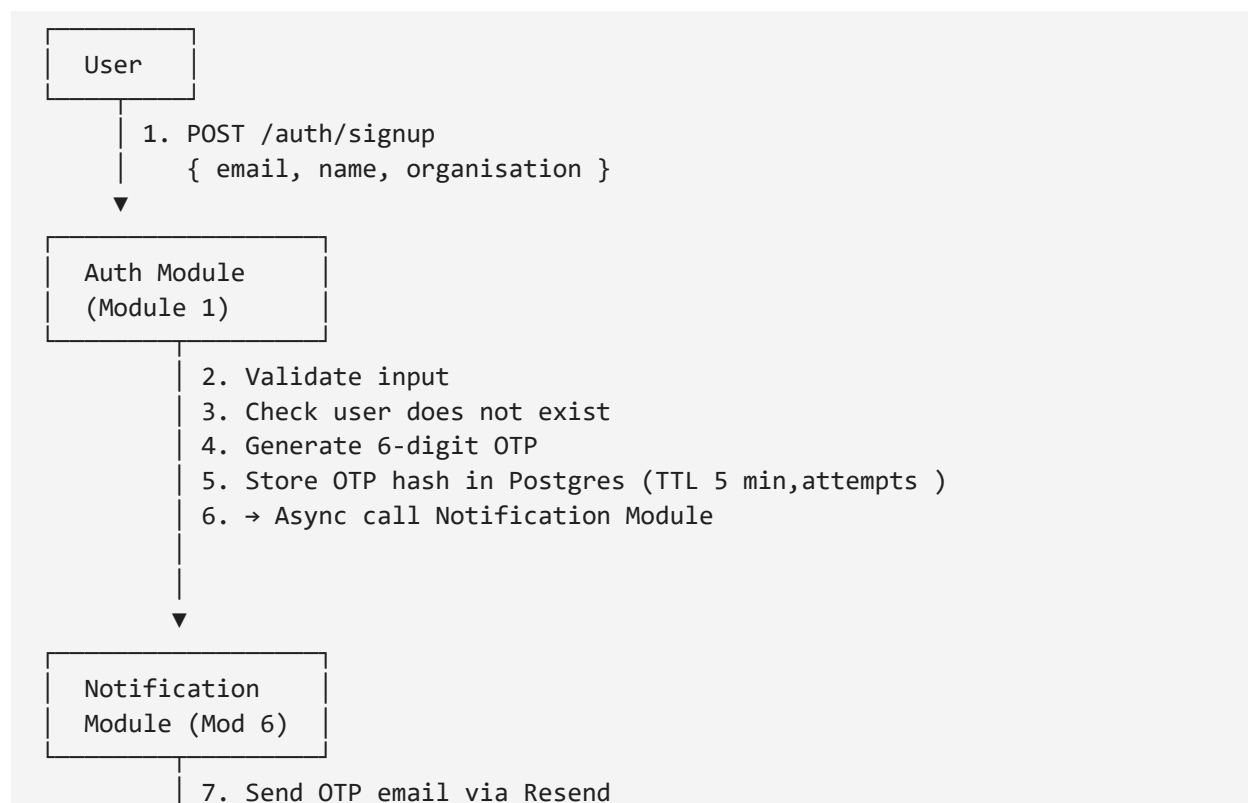
5. Flow 1 — Trust & Verification Flow

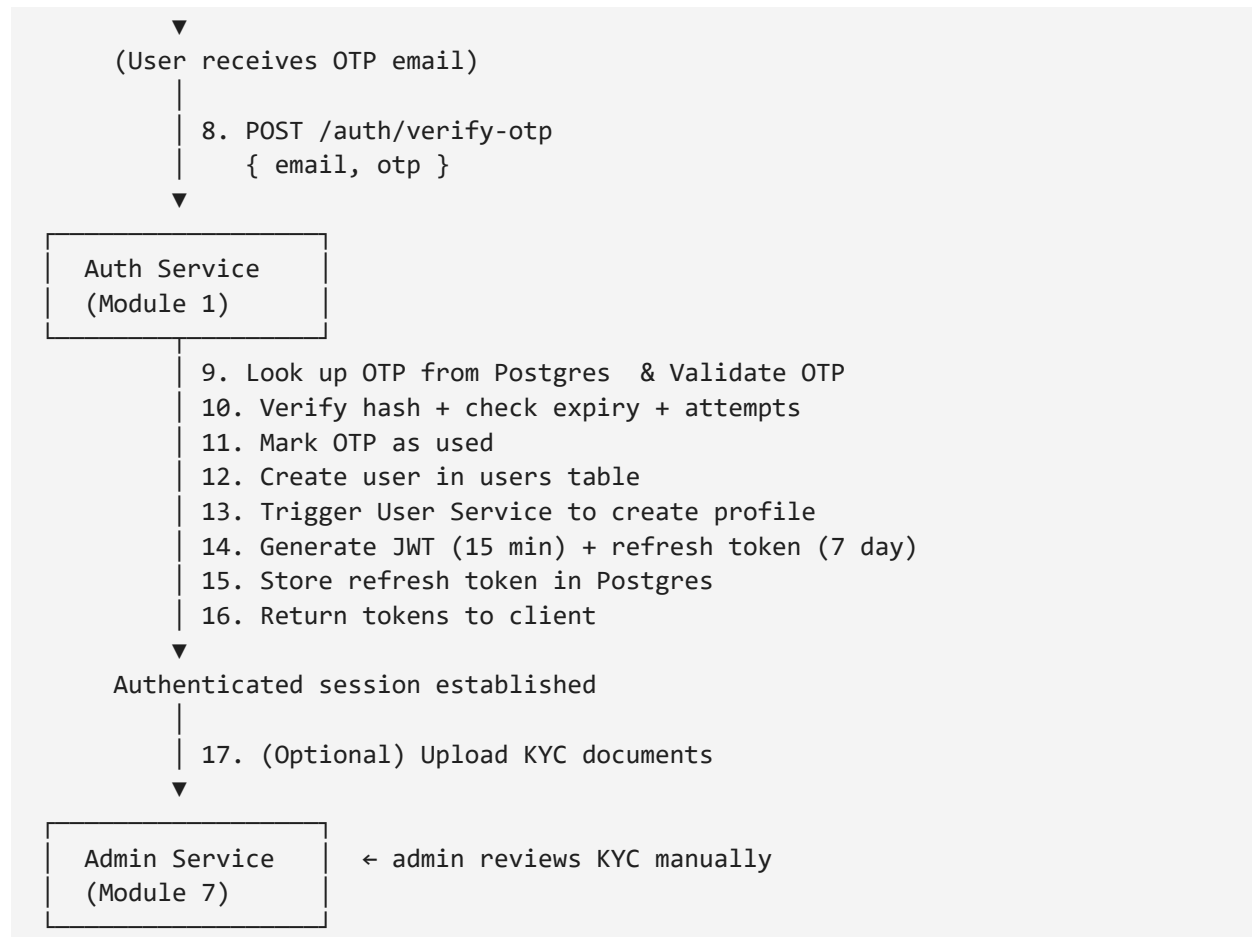
The Trust & Verification Flow is the gateway to the platform. Every NGO user who wants to contribute climate data, submit queries, or access funding information must first establish identity through this flow. The flow uses email OTP for verification (MVP) and is owned by the Auth Service module.

5.1 Modules Involved

Module	Tech Stack	Responsibility in This Flow
Auth Service (Module 1)	Go + Gin + JWT + Argon2id + Resend	Owns the entire flow — signup, OTP generation, OTP validation, user creation, JWT issuance
User Service (Module 2)	Go + Gin	Creates the organisation and user profile after successful verification
Notification Service (Module 6)	Go + Gin + Resend	Sends OTP email via Resend
Admin Service (Module 7)	Go + Gin	Reviews and approves KYC documents post-verification

5.2 Flow Diagram





5.3 Step-by-Step Specification

Step 1 — User Initiates Signup

Endpoint:

POST `/auth/signup`

Request body:

```
{
  "email":      "user@ngo.org",
  "name":       "Shweta Kumar",
  "organisation": "TRDSW",
  "role":       "ngo" // or "donor"
}
```

Owned by:

- Module 1 — Auth Service

Tech used:

- Go + Gin (HTTP routing and validation)
- go-playground/validator (input schema)

Step 2 — Input Validation

- Email format check (RFC 5322)
- Required fields present
- Sanitization to prevent injection

Edge cases:

- Invalid email format → HTTP 400
- Missing fields → HTTP 400

Step 3 — Duplicate User Check

```
SELECT id FROM users WHERE email = $1
```

If exists:

- Return HTTP 409 — user already registered

Step 4 — OTP Generation

```
otp := fmt.Sprintf("%06d", crypto/rand.Intn(900000) + 100000)
```

- 6-digit numeric code
- Generated using crypto/rand (not math/rand) for security

Step 5 — OTP Storage in Postgres**Table schema:**

```
CREATE TABLE otp_codes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL,
  code_hash   TEXT NOT NULL,      -- bcrypt hash of OTP
  expires_at  TIMESTAMPTZ NOT NULL, -- now() + 5 min
  used_at     TIMESTAMPTZ,
  attempt_count INT DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT now()
);
```

```
CREATE INDEX idx_otp_email_active
```

```
ON otp_codes(email, expires_at)
WHERE used_at IS NULL;
```

Cleanup:

- Hourly cron deletes expired OTPs: DELETE FROM otp_codes WHERE expires_at < now() - interval '1 hour'

Step 6 — Trigger Notification Service

- Direct in-process function call (no HTTP)

```
notificationService.SendOTPEmail(email, otp)
```

Step 7 — Send OTP Email (Notification Service / Module 6)**Tech: Resend API**

```
POST https://api.resend.com/emails
Headers: Authorization: Bearer <RESEND_API_KEY>
Body: {
  "from": "noreply@growplus.org",
  "to": ["user@ngo.org"],
  "subject": "Your Grow+ verification code",
  "html": "Your code is <b>123456</b> (valid for 5 minutes)"
}
```

Response to user (HTTP 202):

```
{
  "message": "OTP sent to your email",
  "expires_in": 300
}
```

Step 8 — User Submits OTP**Endpoint:**

```
POST /auth/verify-otp
```

Request body:

```
{
  "email": "user@ngo.org",
  "otp": "123456"
}
```

```
}
```

Steps 9–11 — OTP Validation

```
SELECT id, code_hash, expires_at, attempt_count
FROM otp_codes
WHERE email = $1
      AND used_at IS NULL
      AND expires_at > now()
ORDER BY created_at DESC
LIMIT 1;
```

Validation rules:

- If no row → HTTP 401 (expired or no OTP)
- If attempt_count >= 5 → HTTP 429 (too many attempts)
- If bcrypt.Compare(input, code_hash) fails → increment attempt_count, HTTP 401
- On success → mark used_at = now()

Step 12 — User Creation

Table schema:

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  role        TEXT NOT NULL, -- 'ngo' or 'donor' or 'admin'
  is_verified BOOLEAN DEFAULT TRUE,
  is_kyc_approved BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);
```

Step 13 — User Profile Creation (User Service / Module 2)

```
userService.CreateProfile(userID, organisation, role)
```

Creates row in:

- organisation_profiles or donor_profiles based on role

Steps 14–15 — Token Issuance

Access Token (JWT)

```
{
  "sub": "<user_id>",
  "role": "ngo",
  "exp": <now + 15 min>,
  "iat": <now>
}
```

- Signed using JWT signing key from AWS Secrets Manager
- 15-minute lifetime

Refresh Token (Opaque, stored in Postgres)

```
CREATE TABLE refresh_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  token_hash  TEXT NOT NULL,          -- SHA-256 hash
  expires_at  TIMESTAMPTZ NOT NULL,   -- now() + 7 days
  created_at  TIMESTAMPTZ DEFAULT now(),
  revoked_at  TIMESTAMPTZ,
  rotated_to  UUID REFERENCES refresh_tokens(id)
);

CREATE INDEX idx_refresh_user_active
ON refresh_tokens(user_id)
WHERE revoked_at IS NULL;
```

- Opaque random 32-byte token, hashed with SHA-256 before storage
- 7-day lifetime
- Rotated on use — old token revoked, new token issued
- Supports force-logout (revoke all tokens for a user)

Step 16 — Response to Client

```
HTTP 200 OK
{
  "access_token": "<jwt>",
  "refresh_token": "<opaque>",
  "expires_in": 900,
  "user": {
    "id": "<uuid>",
    "email": "user@ngo.org",
    "role": "ngo"
  }
}
```

```
}  
}
```

Step 17 — KYC Document Upload (Optional, Post-Verification)

Endpoint:

```
POST /auth/kyc/upload
```

- File uploaded to AWS S3 under kyc/{user_id}/
- Record created in kyc_submissions table
- Admin Service (Module 7) reviews via admin dashboard
- On approval, users.is_kyc_approved = true

5.4 Token Refresh Flow

Endpoint:

```
POST /auth/refresh
```

Process:

1. Client sends refresh token
2. Server hashes it (SHA-256) and looks up in refresh_tokens
3. If valid and not revoked, revoke old token (set revoked_at)
4. Issue new access token + new refresh token
5. Link old → new via rotated_to column
6. Return new tokens

Reuse detection (security):

- If a revoked refresh token is presented again → revoke ALL tokens for that user (likely token theft)

5.5 Logout

Endpoint:

```
POST /auth/logout
```

- Sets revoked_at = now() on the user's refresh tokens
- Access token expires naturally within 15 minutes

5.6 Security Considerations

- OTP rate limiting — maximum 5 OTP requests per email per hour
- Failed OTP attempts capped at 5 per OTP code
- All passwords (if added later) hashed with Argon2id
- JWT signed with key from AWS Secrets Manager (auto-rotated annually)
- HTTPS enforced for all endpoints (TLS via AWS ACM)
- CORS restricted to known frontend origins

5.7 Future Phase 2 Additions

- OAuth via Sign in with Google (golang.org/x/oauth2)
- MFA via TOTP (RFC 6238)
- KYC automation via Onfido or Veriff API
- SMS OTP via AWS SNS (when mobile app launches)

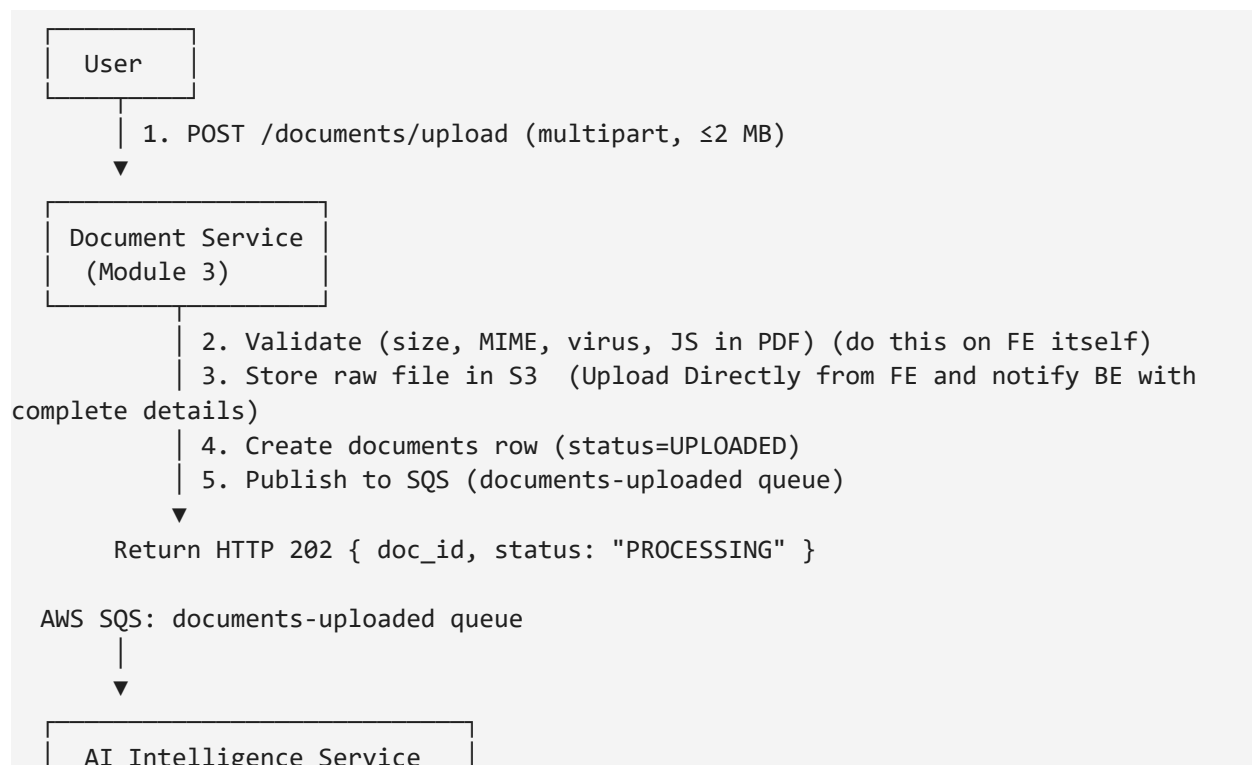
6. Flow 2 — Data Ingestion Flow

The Data Ingestion Flow accepts climate documents uploaded by NGO users, parses their contents, extracts structured climate intelligence using AI, and persists both structured records and vector embeddings into the platform's knowledge base. This knowledge base is then queried by the Climate Twin Matching Flow.

6.1 Modules Involved

Module	Tech Stack	Responsibility in This Flow
Document Service (Module 3)	Go + Gin + AWS S3 SDK	Validates upload, stores file in S3, creates document record, publishes ingestion event to SQS
AI Intelligence Service (Module 5)	Python 3.12 + FastAPI + LiteLLM + PyMuPDF + python-docx + Tesseract + boto3	Consumes SQS event, parses document, extracts climate factors via LLM, generates embeddings, persists to Postgres
Query Service (Module 4)	Go + Gin	Surfaces processing status to frontend via polling endpoint

6.2 Flow Diagram



(Module 5 – Python)
boto3 SQS consumer

6. Fetch file from S3
7. Parse (PyMuPDF / python-docx / chardet)
8. OCR fallback if needed (Tesseract → Gemini)
9. Store extracted text
10. Chunk text
11. Generate embeddings (OpenAI 3-small)
12. LLM extraction (Gemini 2.0 Flash)
→ AQI, rainfall, temperature, location
13. Store in PostgreSQL
 - climate_analysis
 - insights
 - solutions
 - embeddings (PGVector)
14. Refresh global_search_index materialized view
15. Update document status=COMPLETED
16. Delete SQS message (commit)

▼
(User polls GET /documents/{id}/status)

FE → request upload URL

→ upload directly to S3

→ notify backend

6.3 Step-by-Step Specification

Step 1 — Upload Request

Endpoint:

POST /documents/upload (multipart/form-data)

Owned by:

- Module 3 — Document Service

Constraints:

- Max file size: 2 MB
- Allowed types: PDF, DOCX, TXT
- Authorization required: valid JWT

Step 2 — Validation

Performed:

- File size check (rejects > 2 MB)
- MIME type validation via python-magic — checks actual file bytes, not just extension
- PDF JavaScript detection — rejects PDFs containing /JavaScript or /JS markers
- Empty file detection (< 100 bytes)

On failure:

- HTTP 400 (validation error) or HTTP 413 (too large)

Step 3 — Storage in AWS S3**Bucket structure:**

```
ngo-documents/
├── raw/
│   └── {user_id}/{doc_id}/original.{pdf|docx|txt}
├── parsed/
│   └── {user_id}/{doc_id}/extracted.txt
└── exports/
    └── {user_id}/{report_id}/report.pdf
```

Lifecycle:

- Raw files transition to S3 Glacier after 90 days (cost optimisation)

Step 4 — Document Record Creation

```
CREATE TABLE documents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  s3_bucket   TEXT NOT NULL,
  s3_key      TEXT NOT NULL,
  file_name   TEXT NOT NULL,
  file_type   TEXT NOT NULL,      -- 'pdf', 'docx', 'txt'
  file_size   INT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'UPLOADED',
  -- Status lifecycle: UPLOADED → PROCESSING → COMPLETED / FAILED
  extracted_text TEXT,
  page_count  INT,
  text_length INT,
  error_message TEXT,
  error_stage  TEXT,
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now(),
  completed_at TIMESTAMPTZ
);

CREATE INDEX idx_documents_user_status
```

```
ON documents(user_id, status);
```

Step 5 — Publish to SQS

Queue:

- documents-uploaded (visibility timeout: 300s)

Message body:

```
{
  "event_type": "DOCUMENT_UPLOADED",
  "doc_id": "<uuid>",
  "user_id": "<uuid>",
  "s3_bucket": "ngo-documents",
  "s3_key": "raw/<user_id>/<doc_id>/original.pdf",
  "file_type": "pdf",
  "timestamp": "2026-...Z"
}
```

Immediate response to client:

```
HTTP 202 Accepted
{
  "doc_id": "<uuid>",
  "status": "PROCESSING"
}
```

Step 6 — SQS Consumer Picks Up Event

Owned by:

- Module 5 — AI Intelligence Service

Implementation:

```
import boto3
import json
import threading

sqs = boto3.client('sqs')

def consume_documents_uploaded():
    while True:
        msgs = sqs.receive_message(
            QueueUrl=DOCS_UPLOADED_QUEUE,
            MaxNumberOfMessages=10,
            WaitTimeSeconds=20,          # long polling
```

```

    ).get('Messages', [])

    for msg in msgs:
        try:
            body = json.loads(msg['Body'])
            process_document(body)
            sqs.delete_message(
                QueueUrl=DOCS_UPLOADED_QUEUE,
                ReceiptHandle=msg['ReceiptHandle'],
            )
        except Exception as e:
            # Don't delete – SQS will retry per visibility timeout
            logger.error(f"Job failed: {e}")

# Started in background thread alongside FastAPI server
threading.Thread(target=consume_documents_uploaded, daemon=True).start()

```

Step 7 — Document Parsing

Routing by file_type:

File type	Library	Notes
PDF	PyMuPDF (fitz)	Handles ~98% of NGO PDFs without OCR
DOCX	python-docx	Includes explicit table extraction (climate data often in tables)
TXT	Python built-in + chardet	Encoding detection prevents non-UTF-8 failures

PDF parsing example:

```

import fitz

def parse_pdf(file_path: str) -> tuple[str, int]:
    doc = fitz.open(file_path)
    pages = [page.get_text() for page in doc]
    return ('\n\n'.join(pages), len(pages))

```

Step 8 — OCR Fallback (Scanned Documents Only)

Triggered when:

- PDF has < 50 characters of extracted text per page (likely scanned)

Fallback chain:

7. Tesseract (English) — free, handles 80% of clean scans
8. If Tesseract confidence < 70 OR output appears garbled → Gemini 2.0 Flash with image input

Gemini OCR call:

```

from litellm import completion

response = completion(
    model="gemini/gemini-2.0-flash",
    messages=[{
        "role": "user",
        "content": [
            {"type": "text", "text": "Extract all text from this image."},
            {"type": "image_url", "image_url": {"url": image_data_uri}}
        ]
    }]
)

```

Step 9 — Store Extracted Text

```

UPDATE documents
SET extracted_text = $1,
    text_length    = $2,
    page_count     = $3,
    updated_at     = now()
WHERE id = $4;

```

Step 10 — Text Chunking

Strategy:

- Split into ~500-token chunks with 50-token overlap
- Preserve sentence boundaries
- Each chunk gets its own embedding

Step 11 — Embedding Generation

Model: OpenAI text-embedding-3-small

- 1536 dimensions
- Batched calls (up to 100 chunks per API request)
- Cost: \$0.02 per 1M tokens

Embeddings table:

```

CREATE TABLE embeddings (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    doc_id      UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    chunk_text  TEXT NOT NULL,
    embedding   VECTOR(1536) NOT NULL,

```

```

    created_at TIMESTAMPTZ DEFAULT now()
);

-- HNSW index for fast cosine similarity
CREATE INDEX idx_embeddings_hnsw
ON embeddings USING hnsw (embedding vector_cosine_ops);

```

Step 12 — LLM Climate Factor Extraction

Model: Gemini 2.0 Flash (cheapest tier of tiered LLM strategy)

Prompt structure:

```

system: You are a climate intelligence extractor.
       Extract structured climate data from the provided text.

user: Extract the following as JSON:
      - aqi (number, µg/m³)
      - rainfall_mm (number)
      - temperature_c (number)
      - location (text)
      - confidence (0-1)
      - insights (list of short strings)
      - solutions (list of short strings)

      TEXT:
      {extracted_text}

```

LiteLLM call:

```

from litellm import completion

response = completion(
    model="gemini/gemini-2.0-flash",
    messages=[...],
    response_format={"type": "json_object"},
    temperature=0.2,
)

```

Step 13 — Persist Structured Output

Tables:

```

CREATE TABLE climate_analysis (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id       UUID NOT NULL REFERENCES documents(id),
    aqi               NUMERIC,

```



```

rainfall_mm      NUMERIC,
temperature_c    NUMERIC,
location         TEXT,
analysis_confidence NUMERIC,
created_at       TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE insights (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id     UUID NOT NULL REFERENCES documents(id),
  insight_text    TEXT NOT NULL,
  category        TEXT,
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE solutions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id     UUID NOT NULL REFERENCES documents(id),
  solution_text   TEXT NOT NULL,
  similarity_score NUMERIC,
  created_at      TIMESTAMPTZ DEFAULT now()
);

```

Step 14 — Refresh Search Index

After persistence:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY global_search_index;
```

- CONCURRENTLY ensures users can still search while index updates
- Refresh takes ~1–2 seconds at this volume

Step 15 — Mark Document Completed

```

UPDATE documents
SET status      = 'COMPLETED',
    completed_at = now()
WHERE id = $1;

```

Step 16 — Acknowledge SQS Message

- Only after successful persistence — guarantees at-least-once processing
- On failure (any step), message stays in queue and retries up to 3 times
- After 3 failures → moved to documents-uploaded-dlq for human investigation

6.4 Status Polling (Frontend Integration)

Endpoint owned by Query Service (Module 4):

GET /documents/{doc_id}/status

Polling intervals (frontend-driven):

Time since upload	Poll interval
0–15 seconds	Every 3 seconds
15–60 seconds	Every 5 seconds
More than 60 seconds	Every 10 seconds
Stop after	5 minutes (timeout)

Response:

```
{
  "doc_id":    "<uuid>",
  "status":    "PROCESSING" | "COMPLETED" | "FAILED",
  "error":     null | "<message>"
}
```

6.5 Status Lifecycle

```

UPLOADED → PROCESSING → COMPLETED
                |
                → FAILED (with error_message + error_stage)

```

6.6 Error Handling

Failure stage	Behaviour
Validation (Step 2)	HTTP 400 immediately. No queue entry.
S3 upload (Step 3)	Retry 3 times. If still fails → HTTP 500.
Parsing (Step 7)	SQS retry 3x. After DLQ, document.status = FAILED.
Embedding (Step 11)	SQS retry 3x. After DLQ, partial result preserved.
LLM (Step 12)	LiteLLM retries with exponential backoff. After DLQ, document.status = FAILED.
DB persistence (Step 13)	Transaction rolls back. SQS retry.

6.7 Idempotency

- Each event has a unique doc_id

- If a document is already in COMPLETED status when the SQS event arrives, the AI service skips processing and acknowledges the message
- Prevents duplicate processing on SQS at-least-once delivery

6.8 SQS Configuration for This Flow

Setting	Value	Reason
Queue name	documents-uploaded	Single-purpose queue
Type	Standard (not FIFO)	FIFO costs 2x and is unnecessary
Visibility timeout	300 seconds	Parsing + embedding + LLM combined max ~250s
Long polling wait	20 seconds	Reduces empty receives to zero cost
Message retention	4 days	Default; gives time to debug DLQ
Max receive count	3 (then DLQ)	Three retries before human review

7. Flow 3 — Climate Twin Matching Flow

The Climate Twin Matching Flow is the platform's intelligence engine. It accepts a user's climate problem statement, semantically searches the knowledge base built by the Data Ingestion Flow, identifies relevant climate twins (NGOs in similar contexts), and synthesizes actionable recommendations using the tiered LLM strategy.

7.1 Modules Involved

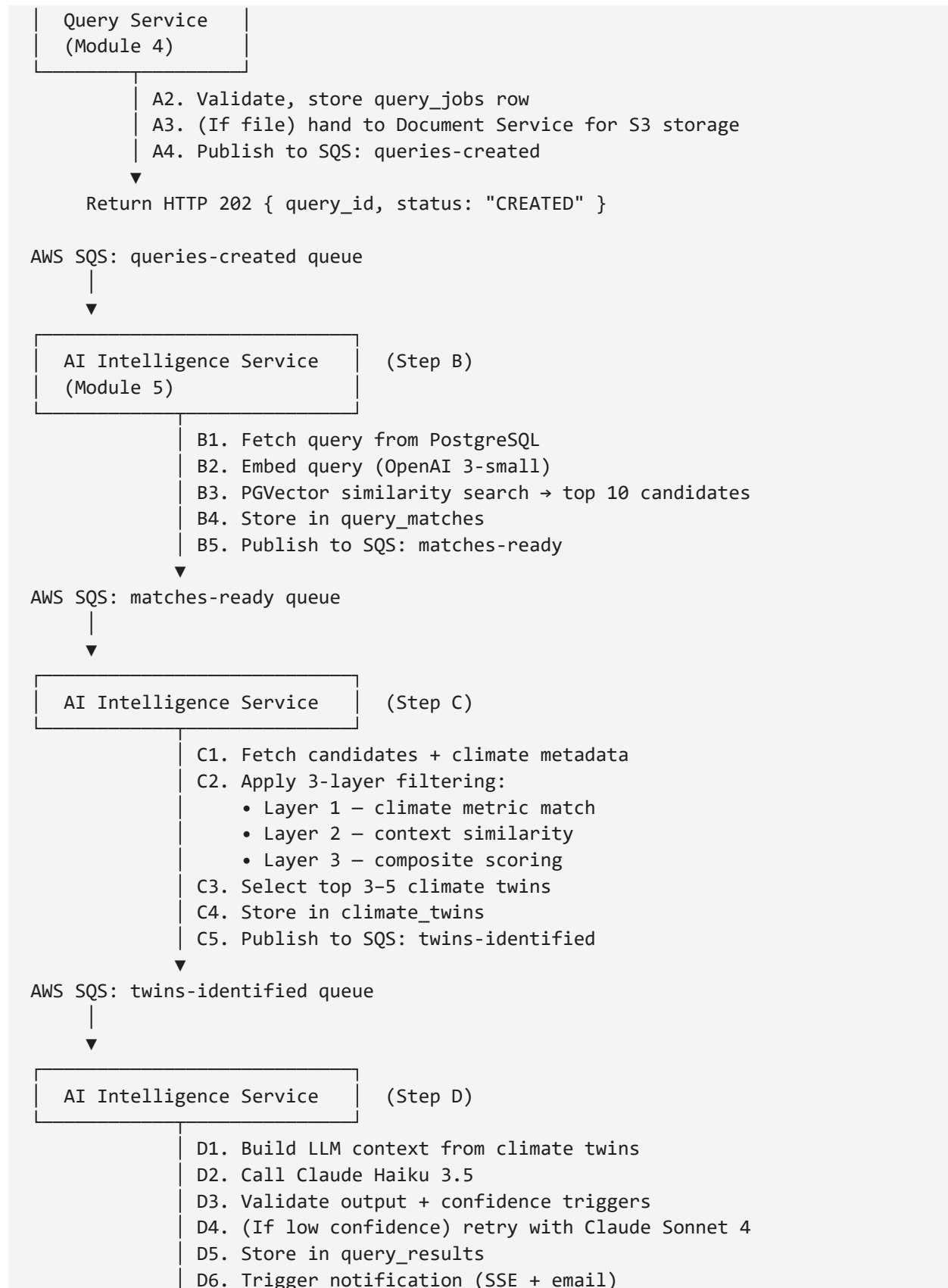
Module	Tech Stack	Responsibility in This Flow
Query Service (Module 4)	Go + Gin	Steps A & E — query submission, status polling, result delivery
AI Intelligence Service (Module 5)	Python 3.12 + FastAPI + LiteLLM + boto3	Steps B–D — embedding, similarity search, twin identification, AI synthesis
Document Service (Module 3)	Go + Gin + AWS S3 SDK	Stores user-uploaded query documents (when query input is a file)
Notification Service (Module 6)	Go + Gin + Resend	Pushes "results ready" notification via SSE / email

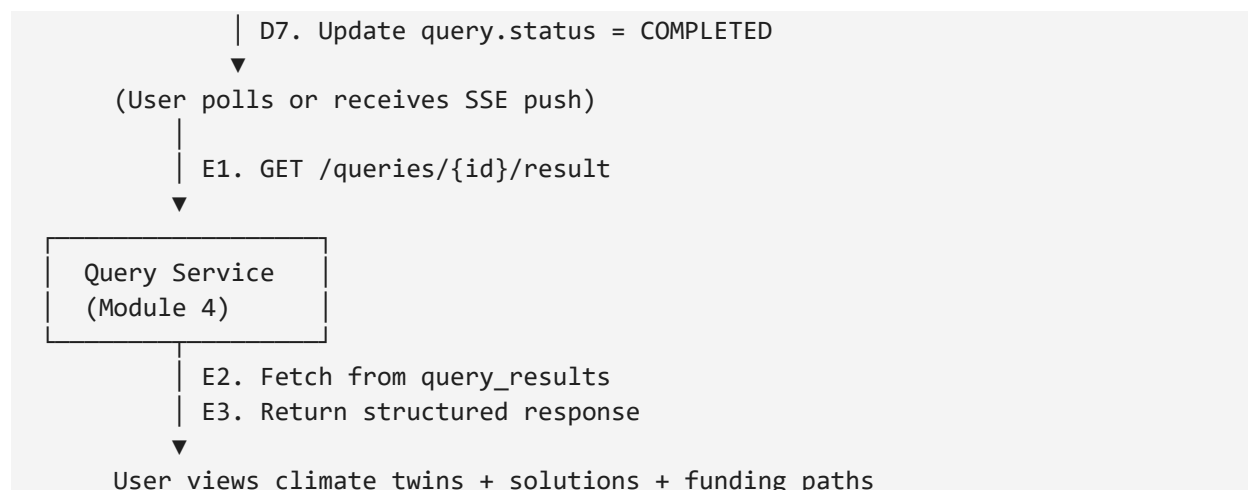
7.2 The Five Steps

Step	Owner	Purpose
A — User Input	Query Service (Module 4)	Capture and store the user's climate problem
B — Similarity Search	AI Intelligence Service (Module 5)	Embed query and find candidate documents via PGVector
C — Climate Twin Identification	AI Intelligence Service (Module 5)	Filter candidates using climate metrics + context
D — AI Synthesis	AI Intelligence Service (Module 5)	Generate solutions, funding paths, recommendations via LLM
E — Result Delivery	Query Service (Module 4)	Return synthesized results to the user

7.3 End-to-End Flow Diagram







7.4 Step A — User Input

Module: Query Service (Module 4)

Tech: Go + Gin

A.1 Endpoint

POST /queries

A.2 Input Modes

Mode	Payload	Use case
Text	{ "input_type": "text", "input_text": "..." }	User describes the problem in their own words
Document	multipart/form-data with file	User uploads a PDF/DOCX describing the problem

A.3 Validation

- Text input: minimum 20 characters, maximum 5,000 characters
- Document input: same constraints as Data Ingestion Flow (≤ 2 MB, allowed types)
- JWT required

A.4 query_jobs Table

```

CREATE TABLE query_jobs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  input_type  TEXT NOT NULL,           -- 'text' or 'document'
  input_text  TEXT,

```

```

document_id  UUID REFERENCES documents(id),
status       TEXT NOT NULL DEFAULT 'CREATED',
-- Lifecycle: CREATED → PROCESSING → MATCH_FOUND →
--            TWINS_IDENTIFIED → COMPLETED / FAILED
error_message TEXT,
created_at   TIMESTAMPTZ DEFAULT now(),
updated_at   TIMESTAMPTZ DEFAULT now(),
completed_at TIMESTAMPTZ
);

CREATE INDEX idx_query_jobs_user_status
ON query_jobs(user_id, status);

```

A.5 Publish to SQS

Queue: queries-created (visibility timeout 60s)

```

{
  "event_type": "QUERY_CREATED",
  "query_id":   "<uuid>",
  "user_id":    "<uuid>",
  "input_type": "text",
  "timestamp":  "...Z"
}

```

A.6 Response

```

HTTP 202 Accepted
{
  "query_id": "<uuid>",
  "status":   "CREATED"
}

```

7.5 Step B — Similarity Search (Engine Room)

Module: AI Intelligence Service (Module 5)

Tech: Python 3.12 + FastAPI + boto3 + OpenAI + PGVector

B.1 Trigger

- SQS consumer reads from queries-created queue
- Updates query_jobs.status = 'PROCESSING'

B.2 Fetch Query Input

```
SELECT input_type, input_text, document_id
FROM query_jobs
WHERE id = $1;
```

If input is a document:

- Fetch parsed text from documents.extracted_text

B.3 Embedding Generation

Model: OpenAI text-embedding-3-small (1536 dims)

```
from openai import OpenAI

client = OpenAI()
embedding = client.embeddings.create(
    model="text-embedding-3-small",
    input=query_text
).data[0].embedding
```

B.4 PGVector Similarity Search

Top-K cosine similarity:

```
SELECT
    doc_id,
    chunk_index,
    chunk_text,
    1 - (embedding <=> $1::vector) AS similarity
FROM embeddings
ORDER BY embedding <=> $1::vector
LIMIT 10;
```

Index used:

- HNSW (vector_cosine_ops) — sub-50ms latency at 200K embeddings

B.5 Store Candidates

```
CREATE TABLE query_matches (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query_id          UUID NOT NULL REFERENCES query_jobs(id),
    doc_id            UUID NOT NULL REFERENCES documents(id),
    chunk_index       INT,
    similarity_score   NUMERIC NOT NULL,
```



```
created_at      TIMESTAMPTZ DEFAULT now()
);
```

B.6 Update Status & Publish Next Event

- Update query_jobs.status = 'MATCH_FOUND'
- Publish to SQS: matches-ready queue

7.6 Step C — Climate Twin Identification

Module: AI Intelligence Service (Module 5)

Tech: Python 3.12 + boto3

C.1 Trigger

- SQS consumer reads from matches-ready queue

C.2 Fetch Climate Metadata for Candidates

```
SELECT
  c.doc_id,
  c.aqi,
  c.rainfall_mm,
  c.temperature_c,
  c.location,
  d.user_id,
  o.organisation_name,
  o.region,
  o.ngo_type,
  qm.similarity_score AS vector_score
FROM query_matches qm
JOIN climate_analysis c ON c.document_id = qm.doc_id
JOIN documents d ON d.id = qm.doc_id
JOIN organisation_profiles o ON o.user_id = d.user_id
WHERE qm.query_id = $1;
```

C.3 Three-Layer Filtering

Layer 1 — Climate Metric Match (hard filter)

```
def climate_match(query_metrics, candidate):
    return (
        abs(query_metrics.aqi - candidate.aqi) <= 0.20 * query_metrics.aqi and
        abs(query_metrics.rainfall_mm - candidate.rainfall_mm) <= 0.25 *
query_metrics.rainfall_mm and
        abs(query_metrics.temperature_c - candidate.temperature_c) <= 0.10 *
```

```
query_metrics.temperature_c
)
```

Layer 2 — Context Similarity Score

Factor	Weight
NGO type match	20%
Region similarity	15%
Problem category match	15%

Layer 3 — Composite Score

```
final_score = (
    vector_score    * 0.50 +
    climate_score   * 0.30 +
    context_score   * 0.20
)
```

C.4 Select Top Twins

- Sort by final_score descending
- Take top 3 to 5 climate twins

C.5 Store Climate Twins

```
CREATE TABLE climate_twins (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query_id          UUID NOT NULL REFERENCES query_jobs(id),
    doc_id            UUID NOT NULL REFERENCES documents(id),
    ngo_user_id       UUID NOT NULL REFERENCES users(id),
    vector_score       NUMERIC,
    climate_score      NUMERIC,
    context_score      NUMERIC,
    final_score        NUMERIC,
    explanation        TEXT,
    is_admin_verified BOOLEAN DEFAULT FALSE,
    created_at        TIMESTAMPTZ DEFAULT now()
);
```

C.6 Update Status & Publish Next Event

- Update query_jobs.status = 'TWINS_IDENTIFIED'
- Publish to SQS: twins-identified queue

7.7 Step D — AI Synthesis (Knowledge Delivery)

Module: AI Intelligence Service (Module 5)

Tech: Python 3.12 + FastAPI + LiteLLM + boto3

D.1 Trigger

- SQS consumer reads from twins-identified queue

D.2 Build LLM Context

```
SELECT
  ct.ngo_user_id, o.organisation_name,
  c.aqi, c.rainfall_mm, c.temperature_c, c.location,
  s.solution_text, i.insight_text
FROM climate_twins ct
JOIN organisation_profiles o ON o.user_id = ct.ngo_user_id
JOIN climate_analysis c ON c.document_id = ct.doc_id
LEFT JOIN solutions s ON s.document_id = ct.doc_id
LEFT JOIN insights i ON i.document_id = ct.doc_id
WHERE ct.query_id = $1
ORDER BY ct.final_score DESC;
```

D.3 Tiered LLM Strategy

Step	Model	Use
Default synthesis	Claude Haiku 3.5	Best fit for user-facing recommendations, 5x cheaper than Sonnet
High-stakes fallback	Claude Sonnet 4	Triggered ONLY when rule-based confidence checks fail

D.4 Synthesis Prompt

system: You are an AI assistant helping NGOs solve climate-related challenges using peer NGO knowledge.

user: User's problem:
{user_input}

Climate twins found:
{twins_data} // structured JSON of top 3-5 NGOs

Generate JSON with:

- summary (2-3 sentences)
- solutions (list of objects: title, description, source_ngo)
- funding_paths (list of objects: donor, amount_range, source_ngo)
- next_steps (list of strings)

- confidence (0-1)

D.5 LLM Call via LiteLLM

```
from litellm import completion

response = completion(
    model="anthropic/claude-haiku-3-5",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt},
    ],
    response_format={"type": "json_object"},
    temperature=0.3,
    max_tokens=1500,
)

result = json.loads(response.choices[0].message.content)
```

D.6 Rule-Based Fallback Triggers

Switch to Claude Sonnet 4 if any of the following is true:

- Top match similarity_score < 0.70 (poor vector match)
- Fewer than 2 climate twins found in Step C
- Document parsing confidence < 80%
- LLM-returned confidence field < 0.75
- User has explicitly flagged a previous result as unhelpful (re-run)

D.7 Persist Result

```
CREATE TABLE query_results (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query_id          UUID NOT NULL REFERENCES query_jobs(id),
    summary           TEXT NOT NULL,
    solutions         JSONB NOT NULL,
    funding_paths     JSONB NOT NULL,
    next_steps        JSONB NOT NULL,
    confidence_score   NUMERIC,
    llm_model_used    TEXT,
    generated_at      TIMESTAMPTZ DEFAULT now()
);
```

D.8 Trigger Notification

In-process function call to Notification Service:

- Insert row in notifications table
- Open SSE connections receive push immediately
- Optional: send email via Resend if user opts in

D.9 Update Final Status

```
UPDATE query_jobs
SET status      = 'COMPLETED',
    completed_at = now()
WHERE id = $1;
```

7.8 Step E — Result Delivery

Module: Query Service (Module 4)

Tech: Go + Gin

E.1 Status Polling Endpoint

```
GET /queries/{query_id}/status
```

Response:

```
{
  "query_id": "<uuid>",
  "status":   "CREATED" | "PROCESSING" | "MATCH_FOUND"
              | "TWINS_IDENTIFIED" | "COMPLETED" | "FAILED"
}
```

E.2 Result Endpoint

```
GET /queries/{query_id}/result
```

E.3 Authorisation

```
WHERE query_id = $1 AND user_id = $2
```

- Prevents IDOR (Insecure Direct Object Reference) attacks

E.4 Response

```
{
  "query_id":    "<uuid>",
  "status":     "COMPLETED",
  "summary":    "Your situation closely matches three NGOs in...",
  "solutions": [
    {
      "title":    "Rainwater harvesting at scale",
      "description": "...",
      "source_ngo": "EcoCare"
    }
  ],
  "funding_paths": [
    {
      "donor":    "IKEA Foundation",
      "amount_range": "$30,000 - $80,000",
      "source_ngo": "EcoCare"
    }
  ],
  "next_steps": ["..."],
  "confidence": 0.91
}
```

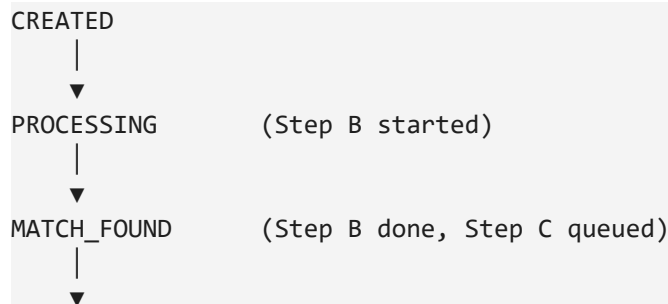
E.5 Real-Time Push (SSE)

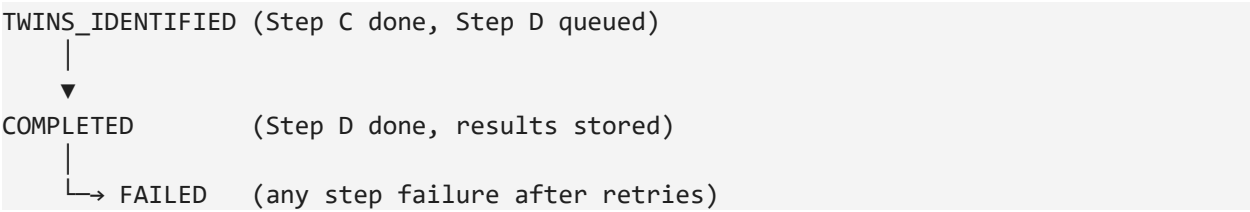
Endpoint:

GET /notifications/stream (text/event-stream)

- Frontend opens SSE connection on app load
- Server pushes 'query_completed' event when query reaches COMPLETED state
- UI updates instantly without polling

7.9 Status Lifecycle (Full)





7.10 SQS Configuration for This Flow

Queue	Purpose	Visibility Timeout
queries-created	Step B — embedding + similarity search	60s
matches-ready	Step C — twin identification + scoring	60s
twins-identified	Step D — LLM synthesis	600s

7.11 Cost Profile (Tiered LLM Strategy)

Daily queries	Strategy	Estimated monthly cost
3,000	Single premium model (e.g. GPT-4)	~\$1,000
3,000	Tiered (Gemini Flash + Haiku + Sonnet fallback)	~\$150

Cost Optimisation Note

The tiered LLM strategy delivers approximately 85% LLM cost reduction with no perceptible quality drop on routine queries. Premium models are reserved for the small fraction (~10%) of cases that genuinely need them.

8. Module Specifications

This section provides per-module specifications including ownership, primary endpoints, owned tables, and integration points. The flows above describe how modules interact at runtime; this section is the per-module reference.

8.1 Module 1 — Auth Service

Field	Detail
Tech	Go + Gin + JWT + Argon2id + Resend
Deployable	api-service
Owned tables	users, otp_codes, refresh_tokens, kyc_submissions
Primary endpoints	POST /auth/signup, POST /auth/verify-otp, POST /auth/login, POST /auth/refresh, POST /auth/logout, POST /auth/kyc/upload
Calls into	User Service (create profile), Notification Service (send OTP email)
Called by	Frontend, Mobile app

8.2 Module 2 — User Service

Field	Detail
Tech	Go + Gin
Deployable	api-service
Owned tables	organisation_profiles, donor_profiles, user_preferences
Primary endpoints	GET /users/me, PUT /users/me, GET /organisations/{id}, PUT /organisations/{id}, GET /donors/{id}
Calls into	(none — leaf module)
Called by	Auth Service (profile creation), Frontend, Search Service

organisation_profiles schema

```
CREATE TABLE organisation_profiles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID UNIQUE NOT NULL REFERENCES users(id),
  organisation_name TEXT NOT NULL,
  region            TEXT,                -- e.g. 'South Asia'
  country            TEXT,
  ngo_type           TEXT,                -- e.g. 'grassroots', 'anchor'
  problem_categories TEXT[],              -- e.g. ['flood', 'agriculture']
  description        TEXT,
  is_kyc_approved    BOOLEAN DEFAULT FALSE,
  created_at         TIMESTAMPTZ DEFAULT now(),
  updated_at         TIMESTAMPTZ DEFAULT now()
);
```


8.3 Module 3 — Document Service

Field	Detail
Tech	Go + Gin + AWS S3 SDK
Deployable	api-service
Owned tables	documents
Primary endpoints	POST /documents/upload, GET /documents/{id}, GET /documents/{id}/status, DELETE /documents/{id}
Publishes to SQS	documents-uploaded queue
Calls into	(none — emits events only)
Called by	Frontend, Mobile, Query Service (when query input is a file)

8.4 Module 4 — Query Service

Field	Detail
Tech	Go + Gin
Deployable	api-service
Owned tables	query_jobs, query_matches, climate_twins, query_results
Primary endpoints	POST /queries, GET /queries, GET /queries/{id}/status, GET /queries/{id}/result, POST /queries/{id}/feedback
Publishes to SQS	queries-created queue
Calls into	Document Service (when input is a file)
Called by	Frontend, Mobile

8.5 Module 5 — AI Intelligence Service

Field	Detail
Tech	Python 3.12 + FastAPI + LiteLLM + PyMuPDF + python-docx + Tesseract + boto3
Deployable	ai-service (separate Fargate service)
Reads tables	documents, query_jobs, embeddings, climate_analysis, query_matches, climate_twins
Writes tables	documents (parsed text), embeddings, climate_analysis, insights, solutions, query_matches, climate_twins, query_results
Consumes from SQS	documents-uploaded, queries-created, matches-ready, twins-identified
External APIs	OpenAI (embeddings), Google Gemini 2.0 Flash, Anthropic Claude (Haiku 3.5, Sonnet 4)
Internal HTTP API	POST /ai/health (used by ALB health checks). All real work is queue-driven.

8.6 Module 6 — Notification Service

Field	Detail
Tech	Go + Gin + Resend
Deployable	api-service

Field	Detail
Owned tables	notifications
Primary endpoints	GET /notifications, GET /notifications/stream (SSE), POST /notifications/{id}/mark-read
Calls into	Resend API (email), FCM (mobile push, future)
Called by	Auth Service (OTP), Document Service (status), Query Service (results), AI Service (via DB notification rows)

notifications schema

```
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  type        TEXT NOT NULL,           -- 'QUERY_COMPLETE', 'TWIN_FOUND' etc.
  payload     JSONB NOT NULL,
  is_read     BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT now(),
  read_at     TIMESTAMPTZ
);
```

```
CREATE INDEX idx_notifications_user
ON notifications(user_id, is_read, created_at DESC);
```

8.7 Module 7 — Admin Service

Field	Detail
Tech	Go + Gin
Deployable	api-service
Owned tables	admin_actions (audit log)
Primary endpoints	GET /admin/kyc/pending, POST /admin/kyc/{id}/approve, POST /admin/kyc/{id}/reject, GET /admin/twins/{query_id}, POST /admin/twins/{query_id}/override, GET /admin/audit-log
RBAC	Requires role = 'admin' on JWT claim
Calls into	User Service (mark KYC approved), Query Service (override climate twins)
Called by	Admin Dashboard (separate React app)

8.8 Module 8 — Search Service

Field	Detail
Tech	Go + Gin + Postgres tsvector + PGVector
Deployable	api-service
Reads tables	Materialized view: global_search_index
Primary endpoints	GET /search?q={query}&type={ngo document solution climate_twin all}&page={n}
Indexes	GIN on search_vector + IVFFlat on embedding column (semantic search)

Field	Detail
	future)
Calls into	(none — read-only)
Called by	Frontend, Mobile, Admin Dashboard

global_search_index materialized view

```

CREATE MATERIALIZED VIEW global_search_index AS
  SELECT
    'ngo' AS content_type,
    u.id AS source_id,
    o.organisation_name AS title,
    o.description AS summary,
    to_tsvector('english',
      coalesce(o.organisation_name, '') || ' ' || coalesce(o.description, '')
    ) AS search_vector
  FROM organisation_profiles o
  JOIN users u ON u.id = o.user_id
  WHERE o.is_kyc_approved = TRUE
UNION ALL
  SELECT
    'document' AS content_type,
    d.id AS source_id,
    d.file_name AS title,
    LEFT(d.extracted_text, 200) AS summary,
    to_tsvector('english',
      coalesce(d.file_name, '') || ' ' || coalesce(d.extracted_text, '')
    ) AS search_vector
  FROM documents d
  WHERE d.status = 'COMPLETED'
UNION ALL
  SELECT
    'solution' AS content_type,
    s.id AS source_id,
    LEFT(s.solution_text, 80) AS title,
    LEFT(s.solution_text, 200) AS summary,
    to_tsvector('english', coalesce(s.solution_text, '')) AS search_vector
  FROM solutions s
UNION ALL
  SELECT
    'climate_twin' AS content_type,
    ct.id AS source_id,
    o.organisation_name AS title,
    coalesce(ct.explanation, '') AS summary,
    to_tsvector('english',
      coalesce(o.organisation_name, '') || ' ' || coalesce(ct.explanation, '')
    ) AS search_vector
  FROM climate_twins ct

```

```
JOIN organisation_profiles o ON o.user_id = ct.ngo_user_id;

CREATE INDEX idx_global_search_vector
ON global_search_index USING GIN(search_vector);

CREATE INDEX idx_global_search_type
ON global_search_index(content_type);
```

Search query

```
SELECT
  content_type, source_id, title, summary,
  ts_rank(search_vector, query) AS rank
FROM global_search_index, plainto_tsquery('english', $1) query
WHERE search_vector @@ query
  AND ($2 = 'all' OR content_type = $2)
ORDER BY rank DESC
LIMIT 20 OFFSET $3;
```

Index refresh

- Refresh triggered by Document Service (after upload completes), User Service (profile update), Query Service (twin identified)

```
REFRESH MATERIALIZED VIEW CONCURRENTLY global_search_index;
```

- CONCURRENTLY ensures users can search while index updates

9. Infrastructure & Deployment

9.1 Cloud Provider

- AWS — single-cloud strategy
- Region: us-east-1 (lowest cost, broadest service availability) — to be confirmed during deployment based on user geographic distribution

9.2 Compute

Service	Tech	Configuration
api-service	AWS ECS Fargate (Go)	0.5 vCPU / 1 GB RAM, 2 tasks min, scale to 10 on CPU > 70%
ai-service	AWS ECS Fargate (Python)	1 vCPU / 4 GB RAM minimum, 1 task min, scale to 4 on SQS queue depth > 50, stopTimeout: 120s

9.3 Networking

Component	Tech	Notes
DNS	Cloudflare DNS	Free DDoS, edge caching, SSL termination
Load balancer	AWS ALB	Routes /api → api-service, /ai → ai-service. ~\$12/mo.
TLS	AWS Certificate Manager	Free, auto-renewing, integrates with ALB
Frontend hosting	Cloudflare Pages	React build deployed to Cloudflare Pages, free tier

9.4 Storage & Database

Component	Tech	Configuration
Primary database	PostgreSQL 16 on AWS RDS	db.t4g.medium (2 vCPU, 4 GB RAM), Multi-AZ, automated backups
Vector search	PGVector (pg_vector extension)	Inside same Postgres, HNSW index in production, ivfflat in dev
Full-text search	Postgres tsvector	Materialized view + GIN indexes
Object storage	AWS S3	Structured buckets: raw/, parsed/, exports/. Lifecycle to Glacier after 90d.
OTP / refresh tokens	PostgreSQL tables	TTL-based via expires_at columns, hourly cleanup cron

9.5 Messaging

Component	Tech	Configuration
Job queue	AWS SQS Standard	4 queues + 4 DLQs, long polling 20s, max receive

Component	Tech	Configuration
		count 3
Queue topology	documents-uploaded · queries-created · matches-ready · twins-identified	Per-queue visibility timeouts (60s/300s/600s by job duration)
Consumer	boto3 inside ai-service	Background threads alongside FastAPI server

9.6 CI/CD & IaC

Component	Tech	Notes
IaC	Terraform	Industry standard, vendor-neutral, mature ecosystem, easier hiring. State stored in S3 with DynamoDB locking.
CI/CD	GitHub Actions or GitLab CI/CD	Two pipelines: deploy-api and deploy-ai (independent). Plus a terraform-plan/apply pipeline for infrastructure.
Container registry	AWS ECR	Lifecycle policy: keep last 10 images per service
AWS auth in CI	OIDC (preferred) or IAM access keys	OIDC eliminates long-lived secrets in CI

9.6.1 Terraform Project Structure

Terraform code is organised into reusable modules and per-environment workspaces. This structure enables clean separation between dev, staging, and production while reusing the same module definitions.

```

infrastructure/
├── modules/                                # Reusable Terraform modules
│   ├── networking/                        # VPC, subnets, security groups
│   ├── ecs-service/                      # Reusable Fargate service definition
│   ├── rds-postgres/                    # RDS instance + parameter groups
│   ├── sqs-queue-with-dlq/              # Standard queue + DLQ pair
│   ├── s3-bucket/                       # Bucket with lifecycle + encryption
│   ├── alb/                             # Application Load Balancer
│   └── secrets/                          # Secrets Manager + SSM parameters
├── environments/
│   ├── dev/
│   │   ├── main.tf                      # Composes modules for dev
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── backend.tf                  # S3 + DynamoDB state
│   ├── staging/
│   │   └── ...
│   └── prod/

```

```

└─ ...
└─ global/                                # One-time / cross-env resources
    ├── ecr-repos.tf
    └── terraform-state-bucket.tf
└─ README.md

```

9.6.2 Terraform State Management

Backend configuration:

```

# environments/prod/backend.tf

terraform {
  backend "s3" {
    bucket      = "growplus-terraform-state"
    key         = "prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
    dynamodb_table = "terraform-state-lock"
  }
}

```

- State stored in dedicated S3 bucket with versioning enabled
- DynamoDB table provides state locking — prevents concurrent terraform apply
- State bucket itself created via global/ Terraform (chicken-and-egg solved by manual bootstrap)
- State files encrypted at rest using AWS KMS

9.6.3 Terraform Modules — Per-Resource Mapping

AWS Resource	Terraform Module	Notes
VPC + subnets	modules/networking	Public + private subnets across 2 AZs
ECS cluster + Fargate services	modules/ecs-service (called twice)	Once for api-service, once for ai-service
ALB + target groups	modules/alb	Routes /api → api-service, /ai → ai-service
RDS Postgres	modules/rds-postgres	db.t4g.medium, Multi-AZ, automated backups
SQS queues (4 of them)	modules/sqs-queue-with-dlq (called 4x)	Each call creates 1 main queue + 1 DLQ
S3 buckets	modules/s3-bucket	ngo-documents bucket with lifecycle policy
Secrets Manager +	modules/secrets	Postgres password, JWT key in Secrets Manager; API keys in SSM

AWS Resource	Terraform Module	Notes
SSM		
IAM roles for ECS tasks	Inside ecs-service module	Task role + execution role per service
CloudWatch log groups	Inside ecs-service module	Created automatically per service
ECR repositories	global/ecr-repos.tf	One repo per deployable, lifecycle policies enabled

9.6.4 Deployment Pipeline for Terraform

Pull Request opened that touches infrastructure/

- terraform-plan job
 1. Authenticate to AWS via OIDC
 2. terraform init
 3. terraform fmt -check
 4. terraform validate
 5. terraform plan -out=tfplan
 6. Post plan as PR comment
- (PR review and approval)
- terraform-apply job (on merge to main)
 1. Authenticate to AWS via OIDC
 2. terraform apply tfplan
 3. Notify Microsoft Teams of changes

- Application deployments (api-service / ai-service) run independently from infrastructure changes
- Application pipelines do NOT run terraform — they only update ECS services with new container images
- This separation prevents accidental infrastructure changes during routine code deploys

9.6.5 Terraform Best Practices for This Project

- Use terraform fmt and tflint in CI to enforce style consistency
- Pin Terraform version in versions.tf (e.g., `>= 1.6.0, < 2.0.0`)
- Pin AWS provider version (e.g., `~> 5.0`)
- Use terraform-docs to auto-generate module documentation
- Tag all resources with project, environment, and managed_by = "terraform" for cost tracking
- Never commit .tfstate files — only state stored in S3 backend
- Use data sources (not hardcoded ARNs) when referencing existing resources

Why Terraform over AWS CDK

Terraform was chosen as the IaC tool because it is the industry standard, offers a simpler mental model (declarative HCL vs TypeScript that generates CloudFormation), provides better debugging visibility into the actual state, and remains vendor-neutral if non-AWS resources (Cloudflare DNS records, Resend webhooks) ever need to be managed alongside AWS infrastructure.

9.7 Secrets Management

Secret	Storage	Reason
Postgres password	AWS Secrets Manager	Auto-rotation justifies the \$0.40/mo cost
JWT signing key	AWS Secrets Manager	Annual rotation
OpenAI API key	AWS SSM Parameter Store	Static, manual rotation only when compromised — free
Anthropic API key	AWS SSM Parameter Store	Static, manual rotation — free
Gemini API key	AWS SSM Parameter Store	Static, manual rotation — free
Resend API key	AWS SSM Parameter Store	Static, manual rotation — free

9.8 Estimated Monthly Cost

Component	Estimated cost (USD/mo)
ECS Fargate (api-service + ai-service)	~\$45
AWS ALB	~\$12
AWS RDS db.t4g.medium	~\$60
AWS ECR	~\$1
AWS Secrets Manager (2 secrets)	~\$1
AWS SSM Parameter Store	Free
AWS SQS (4 queues + 4 DLQs)	<\$2
AWS S3 (storage + transfer)	~\$5
AWS Certificate Manager	Free
AWS CloudWatch (built-in metrics)	Free
Cloudflare DNS + Pages	Free
Resend (email)	~\$2
LLM costs (tiered: Gemini Flash + Claude Haiku + Sonnet fallback)	~\$100–150
Sentry (error tracking, free tier)	Free
Better Stack (logs, uptime, alerting, free tier)	Free
Total estimated monthly	~\$230–280

10. Observability & Operations

The observability stack is intentionally minimal — three tools cover everything a small team needs to operate a 2-service platform reliably.

10.1 Final Pick

Concern	Tool	Notes
Logs + Uptime + Alerting	Better Stack (Logtail) → Microsoft Teams	Single tool covers three concerns. Free tier 1 GB/day. Pipes alerts to a Teams channel.
Error tracking	Sentry → Microsoft Teams	Backend (Go) + frontend (React) errors in one place. Free tier 5K events/mo. Pipes alerts to Teams.
Infrastructure metrics	AWS CloudWatch (built-in)	Free for ECS/RDS/SQS/ALB metrics. Zero setup.
APM / distributed tracing	Skip for MVP	At 2 services communicating via SQS, distributed tracing adds no value. Add Jaeger/Tempo only if service count grows past 10.

10.2 What Each Tool Catches

Failure mode	Better Stack	Sentry
App is down (ECS crash)	✓ Yes	✗ No
Slow response time	✓ Yes	✗ No
Code error / exception	✗ No	✓ Yes
Log pattern match (FATAL, panic)	✓ Yes	✗ No
Frontend JavaScript error	✗ No	✓ Yes
Failed API call to LLM provider	✓ via logs	✓ Yes

10.3 Alert Routing to Microsoft Teams

- Better Stack → Teams via native integration (configured in Better Stack dashboard)
- Sentry → Teams via native integration (configured in Sentry → Settings → Integrations)
- Both tools post structured messages to a dedicated #alerts channel

10.4 Day-to-Day Operations Workflow

Issue reported / alert fires



Check Microsoft Teams #alerts

- If Sentry alert: open Sentry
 - see exact stack trace, line number, affected users, frequency
- If Better Stack alert: open Better Stack
 - search logs around timestamp
 - check uptime history
- If infrastructure issue: open CloudWatch
 - check ECS CPU / memory
 - check RDS connections
 - check SQS queue depth

10.5 Status Lifecycle Telemetry

- Every status transition (UPLOADED → PROCESSING → COMPLETED) emitted as a structured log line
- Better Stack alerts fire if a document or query stays in PROCESSING for > 10 minutes
- CloudWatch tracks SQS queue depth — alerts fire if > 100 messages backlog

11. Security & Compliance

11.1 Authentication & Authorization

- All endpoints (except /auth/signup, /auth/verify-otp, /auth/login) require a valid JWT
- Access tokens: 15-minute lifetime, signed with key from AWS Secrets Manager
- Refresh tokens: opaque random 32-byte tokens, SHA-256 hashed in Postgres, 7-day lifetime, rotated on use
- RBAC: role claim in JWT (ngo, donor, admin) gates module access
- Admin endpoints require role = 'admin' (Module 7 only)
- All resource-scoped queries enforce user_id = \$current_user (prevents IDOR)

11.2 Transport Security

- HTTPS enforced for all public endpoints (TLS via AWS ACM)
- HTTP requests redirected to HTTPS by ALB
- HSTS header set (max-age=31536000)
- CORS restricted to known frontend origins

11.3 Data Protection

- Postgres: encryption at rest (RDS), encryption in transit (TLS)
- S3: server-side encryption with AWS-managed keys (SSE-S3)
- OTPs hashed (bcrypt) before storage — never stored in plaintext
- Refresh tokens hashed (SHA-256) before storage
- Secrets in AWS Secrets Manager / SSM Parameter Store — never in code or environment variables

11.4 Input Validation & Abuse Protection

- All inputs validated via go-playground/validator (Go) and Pydantic (Python)
- File uploads: real MIME type validation via python-magic, not extension trust
- PDF uploads: scanned for /JavaScript markers (rejected if found)
- Rate limiting: 5 OTP requests per email per hour, 60 API requests per minute per user
- File size cap: 2 MB

11.5 Audit & Compliance

- Admin actions logged in admin_actions table with actor, timestamp, target, action

- Login events logged
- Refresh token rotation logged
- Data provenance — every embedding, climate analysis, and synthesised result links back to source document_id
- KYC documents stored separately in S3 with restricted access

11.6 Secrets Management Summary

Secret	Stored in	Access pattern
Postgres password	AWS Secrets Manager	Fetches at api-service / ai-service startup
JWT signing key	AWS Secrets Manager	Fetches at api-service startup
LLM API keys (OpenAI, Anthropic, Gemini)	AWS SSM Parameter Store	Fetches at ai-service startup
Resend API key	AWS SSM Parameter Store	Fetches at api-service startup

12. Critical Operational Considerations

These are non-functional requirements that materially affect production stability and must be implemented from day one.

12.1 Idempotency

- Every SQS event carries a unique identifier (doc_id or query_id)
- Workers check current status before processing — if already terminal (COMPLETED, FAILED), skip and acknowledge
- Prevents duplicate processing on at-least-once delivery

12.2 Retry & Dead Letter Queue

Failure	SQS retry behaviour	DLQ destination
Parsing error	3 retries with exponential backoff	documents-uploaded-dlq
LLM API timeout / rate limit	LiteLLM internal retry → SQS retry	queries-created-dlq / twins-identified-dlq
DB transaction conflict	SQS retry (transient)	Per-queue DLQ
DLQ investigation	Manual via admin dashboard	Daily review

12.3 Status Lifecycle (System-Wide)

Documents:

UPLOADED → PROCESSING → COMPLETED / FAILED

Queries:

CREATED → PROCESSING → MATCH_FOUND → TWINS_IDENTIFIED
→ COMPLETED / FAILED

12.4 Error Recording

- documents.error_message and documents.error_stage capture parsing/AI failures
- query_jobs.error_message captures query pipeline failures
- Sentry captures every exception with full context

12.5 Rate Limiting

- Per-user: 60 API requests per minute
- Per-email OTP: 5 requests per hour
- Per-IP signup: 10 per hour
- Implemented as middleware in Go using token bucket algorithm

12.6 Data Lifecycle

- Raw S3 documents: transition to Glacier after 90 days
- Expired OTPs: hourly cleanup cron
- Revoked refresh tokens: weekly cleanup cron (after 30-day audit window)
- Failed query_jobs: weekly cleanup cron (after 30-day audit window)

12.7 Backup & Disaster Recovery

- RDS automated backups: 7-day retention
- RDS Multi-AZ: automatic failover
- S3: cross-region replication (production)
- Recovery Point Objective (RPO): < 5 minutes
- Recovery Time Objective (RTO): < 1 hour

13. Production Hardening

This section consolidates the production hardening updates introduced in the LLD. Each subsection addresses a specific reliability, security, or operational gap identified and specifies the implementation pattern in detail. These updates **MUST** be implemented before the platform is opened to real NGO users.

Authority Note — Sections 13.1 through 13.10 supersede or extend the corresponding patterns in earlier sections of this document. Where a conflict exists between an earlier section and Section 13, Section 13 is authoritative.

13.1 Failure Strategy — Retry, DLQ, and Failure State Tracking

The platform performs many asynchronous operations: parsing, embedding generation, climate factor extraction, similarity search, and AI synthesis. Any of these can fail transiently (network, API timeout) or permanently (corrupted document, hard validation error).

13.1.1 Retry Policy

Every SQS-driven worker follows this contract:

- Maximum retry attempts: 3 (matches existing SQS Max Receive Count)
- Backoff: SQS visibility timeout doubles per receive (handled by AWS automatically)
- Transient errors (timeout, 5xx, network) → release message back to queue (do NOT delete)
- Permanent errors (validation, 4xx, malformed) → mark FAILED in DB and delete from queue immediately
- After 3 receives → message lands in DLQ for human review

13.1.2 Schema Updates — `retry_count` and `error_reason`

The `documents` and `query_jobs` tables are extended with explicit retry tracking columns. These columns are written by the worker on every retry attempt so operators can see the exact failure trail without reading SQS message logs.

```
-- documents table --
ALTER TABLE documents
  ADD COLUMN IF NOT EXISTS retry_count INT NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS error_reason TEXT, -- short machine code (e.g.
  PARSE_TIMEOUT)
  ADD COLUMN IF NOT EXISTS error_stage TEXT, -- which step failed
  (PARSE/EMBED/EXTRACT/PERSIST)
  ADD COLUMN IF NOT EXISTS last_error_at TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS reviewed_by UUID REFERENCES users(id),
  ADD COLUMN IF NOT EXISTS reviewed_at TIMESTAMPTZ;
```



```
-- query_jobs table
ALTER TABLE query_jobs
  ADD COLUMN IF NOT EXISTS retry_count INT NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS error_reason TEXT,
  ADD COLUMN IF NOT EXISTS error_stage TEXT,
SIMILARITY/MATCH/SYNTHESIS
  ADD COLUMN IF NOT EXISTS last_error_at TIMESTAMPTZ;

-- Index for the operations dashboard ("show failed in last 24h")
CREATE INDEX IF NOT EXISTS idx_documents_failed
  ON documents(status, last_error_at DESC) WHERE status = 'FAILED';

CREATE INDEX IF NOT EXISTS idx_query_jobs_failed
  ON query_jobs(status, last_error_at DESC) WHERE status = 'FAILED';
```

13.1.3 Worker Implementation Pattern

Every SQS consumer wraps its work in this canonical pattern:

```
def process_with_retry(msg):
    body = json.loads(msg["Body"])
    doc_id = body["doc_id"]
    receive_count = int(msg["Attributes"]["ApproximateReceiveCount"])

    try:
        # Idempotency check – see Section 13.4
        if already_terminal(doc_id):
            ack(msg)
            return

        process_document(body)          # the actual work
        ack(msg)                        # delete from queue ONLY on success

    except PermanentError as e:
        # 4xx, validation, corrupted file – never retry
        update_doc(doc_id,
            status='FAILED',
            error_reason=e.code,
            error_stage=e.stage,
            last_error_at=now())
        ack(msg)                        # remove from queue, don't retry
        sentry.capture_exception(e)

    except TransientError as e:
        # 5xx, timeout, network – let SQS retry
        update_doc(doc_id,
            retry_count=receive_count,
            error_reason=e.code,
            error_stage=e.stage,
            last_error_at=now())
```

```
# NO ack – message returns to queue after visibility timeout
# On 4th receive, AWS routes to DLQ automatically
```

13.1.4 Status Lifecycle

Documents and queries gain an explicit NEEDS_REVIEW state for AI-uncertain results:

Documents:

```
UPLOADED -> PROCESSING -> COMPLETED
      \-> NEEDS_REVIEW    (low AI confidence – see 13.3)
      \-> FAILED          (terminal failure)
```

Queries:

```
CREATED -> PROCESSING -> MATCH_FOUND -> TWINS_IDENTIFIED -> COMPLETED
      \-> NEEDS_REVIEW
      \-> FAILED
```

13.1.5 DLQ Operations

Queue	DLQ	Review SLA
documents-uploaded	documents-uploaded-dlq	Daily review by operations engineer
queries-created	queries-created-dlq	Same-day if user-impacting
matches-ready	matches-ready-dlq	Daily
twins-identified	twins-identified-dlq	Daily

Better Stack alert fires immediately when any DLQ depth > 0. The operations dashboard (Module 7 — Admin Service) exposes a 'Failed Items' view showing all FAILED documents and queries with their error_reason, error_stage, retry_count, and a 'Replay' action that re-enqueues the message.

13.2 Observability — Structured Logs, Metrics, and Alerts

Section 10 specifies the observability tool stack (Better Stack + Sentry + CloudWatch). It specifies the contract that application code must follow so those tools deliver actionable signal.

13.2.1 Structured JSON Logs (Mandatory)

Every log line emitted by api-service and ai-service MUST be a single line of JSON conforming to the schema below. Plain-text logs are not permitted in production code paths.

```
{
  "timestamp": "2026-05-04T09:30:14.812Z",
  "level":     "INFO" | "WARN" | "ERROR",
  "service":   "api-service" | "ai-service",
  "module":    "auth" | "document" | "query" | "ai-intelligence" | "...",
  "request_id": "<uuid – propagated via X-Request-ID header end-to-end>",
  "user_id":   "<uuid or null>",
```

```

"doc_id":      "<uuid or null>",
"query_id":    "<uuid or null>",
"event":       "DOCUMENT_PARSE_START" | "LLM_CALL" | "DLQ_LANDED" | ...,
"duration_ms": 1842,
"error_code":  "PARSE_TIMEOUT" | null,
"message":     "Human-readable detail"
}

```

Implementation:

- Go: logrus or zerolog with JSON formatter; request_id from middleware
- Python: structlog with JSONRenderer; request_id from FastAPI middleware
- Both services pipe stdout to Better Stack via the Logtail Vector agent

13.2.2 Metrics — Already Free via CloudWatch

AWS publishes the following metrics free of charge for the services this platform uses. No Prometheus or Grafana installation is needed at this scale — adding them would create operational overhead that contradicts the platform's cost-first principle. Revisit only if service count grows past 10.

Source	Key Metric	Alert Threshold
ECS Fargate	CPUUtilization (api-service)	> 80% for 10 min
ECS Fargate	MemoryUtilization (ai-service)	> 85% for 5 min
RDS	DatabaseConnections	> 80% of max_connections
RDS	CPUUtilization	> 75% sustained 15 min
SQS	ApproximateNumberOfMessagesVisible	> 100 (any queue)
SQS DLQ	ApproximateNumberOfMessagesVisible	> 0 (immediate alert)
ALB	HTTPCode_Target_5XX_Count	> 1% of total requests
ALB	TargetResponseTime	P95 > 1s for 5 min

13.2.3 Application-Level Metrics (CloudWatch Custom Metrics)

Workers push these to CloudWatch via the embedded metric format (no extra cost for first 1M data points):

- document_parse_duration_ms (histogram, dimensions: file_type)
- llm_call_duration_ms (histogram, dimensions: model, success/fail)
- ai_confidence_score (histogram — see 13.3 — alerts if P50 < 0.7)
- twins_match_count (histogram — alerts if P50 < 2 over 1 hour, indicates poor recall)

13.2.4 Alert Routing

All alerts route to the #grow-alerts Microsoft Teams channel. Severity routing:

Severity	Trigger	Channel
P1 — page on-call	Service down, DLQ > 0, RDS unreachable	Teams + email + SMS via Better Stack escalation
P2 — same-day	P95 latency breach, high error rate	Teams
P3 — daily digest	Capacity warnings, cost anomalies	Teams summary 09:00 IST daily

13.3 AI Layer Reliability — Validation, Confidence, and Human Review

LLMs are non-deterministic and can hallucinate. We already specified a tiered LLM strategy and rule-based fallback (Section 7.7 D.6). We are adding an explicit post-LLM validation layer and human-in-the-loop review path.

13.3.1 Schema Validation (Hard Gate)

Every LLM JSON response is validated against a Pydantic schema BEFORE being trusted. If the LLM returns malformed JSON, missing required fields, or values outside expected ranges, the call is treated as a transient failure and retried (up to 2 retries within the same SQS attempt).

```
from pydantic import BaseModel, Field, ValidationError

class ClimateExtraction(BaseModel):
    aqi: float | None = Field(ge=0, le=1000)
    rainfall_mm: float | None = Field(ge=0, le=10000)
    temperature_c: float | None = Field(ge=-60, le=60)
    location: str | None
    confidence: float = Field(ge=0, le=1)
    insights: list[str] = Field(max_length=20)
    solutions: list[str] = Field(max_length=20)

def call_extraction_llm(text: str) -> ClimateExtraction:
    for attempt in range(3):
        raw = litellm.completion(...).choices[0].message.content
        try:
            return ClimateExtraction.model_validate_json(raw)
        except ValidationError as ve:
            log.warn("llm_schema_invalid", attempt=attempt, error=str(ve))
            if attempt == 2:
                raise PermanentError("LLM_INVALID_SCHEMA", stage="EXTRACT")
```

13.3.2 Confidence Threshold and NEEDS_REVIEW

When the LLM returns a confidence < 0.75, the result is persisted but the document or query is marked NEEDS_REVIEW (rather than COMPLETED). NEEDS_REVIEW items appear in the admin dashboard for

human verification before they enter the searchable knowledge base.

```
CONFIDENCE_THRESHOLD = 0.75

if extraction.confidence < CONFIDENCE_THRESHOLD:
    update_doc(doc_id,
        status='NEEDS_REVIEW',
        error_stage='LOW_CONFIDENCE',
        error_reason=f'AI confidence {extraction.confidence:.2f} <
{CONFIDENCE_THRESHOLD}')
    notify_admin(doc_id, reason='LOW_CONFIDENCE')
else:
    persist_extraction(doc_id, extraction)
    update_doc(doc_id, status='COMPLETED')
```

13.3.3 Cross-Field Sanity Checks

Beyond schema validation, the application performs domain sanity checks. These catch hallucinations where the JSON is structurally valid but semantically nonsensical (e.g. AQI of 50,000).

- AQI must be 0–1000 (PM2.5 typical worst-case 800)
- Rainfall (mm/year) must be 0–10,000 (Cherrapunji record ~12,000)
- Temperature must be –60 to +60 °C
- If location field is non-empty, it must contain at least one alphabetic character
- Sanity check failures → status = NEEDS_REVIEW with error_reason = 'SANITY_FAIL_<field>'

13.3.4 Human-in-the-Loop Review (Module 7 — Admin Service)

Module 7 (Admin Service) gains a new 'AI Review Queue' view. Admins can: (a) approve the AI extraction as-is, (b) edit individual fields and approve, or (c) reject the document (marks it FAILED with reviewer-supplied reason).

```
-- New table – review actions audit trail
CREATE TABLE ai_reviews (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id       UUID REFERENCES documents(id),
    query_id          UUID REFERENCES query_jobs(id),
    reviewer_id       UUID NOT NULL REFERENCES users(id),
    decision          TEXT NOT NULL CHECK (decision IN ('APPROVED', 'EDITED',
'REJECTED')),
    edits             JSONB,                    -- field-level diff if EDITED
    reason            TEXT,                    -- if REJECTED
    reviewed_at       TIMESTAMPTZ DEFAULT now()
);
```

13.4 Idempotency — Preventing Duplicate Work

AWS SQS guarantees at-least-once delivery — every message can arrive 1+ times. We already addressed this via terminal-status checks (Section 6.7). It formalises the pattern as a single 'idempotency_key'

contract that every worker follows.

13.4.1 Idempotency Key on Every Event

All SQS message bodies now carry an explicit `idempotency_key` field (in addition to the entity ID):

```
{
  "event_type":      "DOCUMENT_UPLOADED",
  "idempotency_key": "doc:b7e4...:v1",    // entity-id + version qualifier
  "doc_id":          "<uuid>",
  "user_id":         "<uuid>",
  "s3_bucket":       "ngo-documents",
  "s3_key":          "raw/<user_id>/<doc_id>/original.pdf",
  "file_type":       "pdf",
  "timestamp":       "2026-05-04T09:30:14.812Z"
}
```

13.4.2 Two-Layer Idempotency Check

The worker performs the check in two cheap steps before doing any real work:

```
def already_processed(idempotency_key, doc_id):
    # Layer 1 – entity-status check (almost always sufficient)
    row = db.fetch_one("SELECT status FROM documents WHERE id = %s", doc_id)
    if row and row["status"] in ("COMPLETED", "FAILED", "NEEDS_REVIEW"):
        return True

    # Layer 2 – idempotency log table (catches in-flight duplicates)
    existing = db.fetch_one(
        "SELECT 1 FROM processed_events WHERE idempotency_key = %s",
        idempotency_key)
    return existing is not None

def mark_processed(idempotency_key):
    db.execute("""
        INSERT INTO processed_events (idempotency_key, processed_at)
        VALUES (%s, now())
        ON CONFLICT (idempotency_key) DO NOTHING
    """, idempotency_key)
```

13.4.3 Schema — `processed_events` Table

```
CREATE TABLE processed_events (
    idempotency_key TEXT PRIMARY KEY,
    processed_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Cleanup cron – weekly delete of rows older than 30 days
DELETE FROM processed_events
WHERE processed_at < now() - INTERVAL '30 days';
```

Design Rationale — Why both layers? Layer 1 is fast and covers 99% of duplicates. Layer 2 protects

against the rare race where a worker is mid-processing and a duplicate arrives — without it, two parallel workers could write to `documents.embeddings` twice for the same chunk.

13.5 File Storage — Pre-Signed URL Direct Upload to S3

Initially we specified that the client uploads to `api-service` which then forwards to S3 (Section 6.3, Step 1–3). We are changing this to direct browser-to-S3 upload via S3 pre-signed URLs. This removes the `api-service` from the upload bandwidth path entirely — ECS instances no longer carry user file bytes.

13.5.1 Why This Matters

Concern	Impact of Production Hardening Change
api-service CPU	No longer parses multipart bodies. ~30% headroom recovered.
ECS scaling	api-service scales on request count, not byte volume. Smoother.
Upload speed (Shweta on 3G)	Direct S3 connection uses S3 Transfer Acceleration. ~2× faster.
Cost	No ALB data-out charges for upload bytes. Saves ~\$5/mo at projected volumes.
Failure handling	Browser retries S3 directly. api-service never sees a half-uploaded file.

13.5.2 New Three-Call Upload Flow

Old:

```
Client -> [POST /documents/upload, multipart 2MB] -> api-service -> S3
(api-service holds the file in memory)
```

Now:

- Client -> [POST /documents/upload-url, JSON metadata] -> api-service
api-service:
 - Validates metadata (`file_name`, `file_size`, `file_type`, `content_md5`)
 - Reserves a `doc_id`, inserts documents row with `status=AWAITING_UPLOAD`
 - Generates pre-signed PUT URL (15-minute expiry, content-length-range condition)
 api-service -> Client:


```
{ "doc_id": "<uuid>", "upload_url": "https://...", "expires_at": "..."}
    
```
- Client -> [PUT direct to S3 with the pre-signed URL]
S3 stores the file. Returns 200 to client.
- Client -> [POST /documents/{doc_id}/finalize] -> api-service
api-service:
 - Verifies S3 object exists at the expected key
 - Verifies size matches reservation (refuse if mismatched)
 - Updates `documents.status = UPLOADED`
 - Publishes `DOCUMENT_UPLOADED` event to SQS

```
api-service -> Client: { "doc_id": "...", "status": "PROCESSING" }
```

13.5.3 Pre-Signed URL Generation (Go)

```
func GenerateUploadURL(userID, docID uuid.UUID, fileName string, fileSize int64)
(string, error) {
    s3Client := s3.NewPresignClient(s3.NewFromConfig(awsCfg))

    key := fmt.Sprintf("raw/%s/%s/original.%s", userID, docID, fileExt(fileName))

    req, err := s3Client.PresignPutObject(ctx, &s3.PutObjectInput{
        Bucket:      aws.String("ngo-documents"),
        Key:         aws.String(key),
        ContentLength: aws.Int64(fileSize),
        Metadata: map[string]string{
            "user-id": userID.String(),
            "doc-id": docID.String(),
        },
    }, func(po *s3.PresignOptions) {
        po.Expires = 15 * time.Minute
    })
    if err != nil { return "", err }
    return req.URL, nil
}
```

13.5.4 S3 Bucket Policy — Hardening

Pre-signed URLs do not bypass bucket-level controls. Bucket policy enforces:

- Block all public access (S3 'Block Public Access' settings ON)
- Server-side encryption mandatory (SSE-S3 minimum, SSE-KMS for /kyc/ prefix)
- PUT requests denied unless they carry the application's signature (no anonymous uploads possible)
- Lifecycle: raw/* objects without an associated documents row in COMPLETED/UPLOADED state are deleted after 24h (cleans up abandoned uploads)

13.5.5 CORS Configuration

The S3 bucket gains CORS configuration allowing PUT from the application origin only:

```
[
  {
    "AllowedOrigins": ["https://app.growplus.org",
"https://staging.growplus.org"],
    "AllowedMethods": ["PUT"],
    "AllowedHeaders": ["Content-Type", "Content-MD5", "x-amz-meta-*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```


13.6 Real-Time Result Delivery — SSE Primary, Polling Fallback

Note: Section 7.8 (E.5 Real-Time Push) already specifies Server-Sent Events (SSE) as the primary delivery mechanism for query results, with HTTP polling as the fallback. It reaffirms this design — SSE is NOT a future enhancement, it ships in MVP.

13.6.1 Why SSE, Not WebSockets

Concern	SSE Outcome
Direction needed	Server → client only — perfect fit for status push
Infrastructure	Standard HTTP (works behind ALB without sticky sessions or upgrade handling)
Browser support	Native EventSource — no library dependency on the frontend
Reconnection	Built into the protocol with Last-Event-ID resume
3G/low bandwidth	Single open connection, much lighter than WebSocket frame overhead
WebSocket alternative	Reserved for Phase 2 if bidirectional real-time chat features are added

13.6.2 Polling as Fallback

The frontend opens an SSE connection. If the connection fails or the browser does not support EventSource, the frontend automatically degrades to HTTP polling at the intervals specified in Section 6.4 (3s/5s/10s tiered).

13.7 Rate Limiting and Abuse Protection (Strengthened)

Sections 11.4 and 12.5 specify per-user and per-email rate limits. It adds per-resource and per-day quotas to defend against credential-stuffed abuse and runaway costs.

13.7.1 Complete Rate Limit Matrix

Limit Scope	Threshold	Implementation
API requests / user	60 per minute	Token bucket middleware (Go) — already present
OTP requests / email	5 per hour	Already present
Signups / IP	10 per hour	Already present
Document uploads / user / day	20 per day	With this Production Hardening Step — counted via documents created_at
Document uploads / user /	5 per hour	With this Production Hardening Step — burst protection

hour		
Queries / user / day	100 per day	With this Production Hardening Step — protects LLM cost ceiling
Queries / user / minute	10 per minute	With this Production Hardening Step — burst protection
Pre-signed URL requests / user / hour	10 per hour	With this Production Hardening Step — limits upload reservation churn
KYC document uploads / user	5 lifetime (admin can reset)	With this Production Hardening Step — anti-DoS on KYC review queue

13.7.2 Implementation

- Per-minute / per-hour limits: in-memory token bucket via go-chi/httptrate (already in api-service)
- Per-day limits: SQL count() on documents/query_jobs filtered by user_id and created_at >= now() - INTERVAL '1 day'
- On limit breach: HTTP 429 with Retry-After header indicating reset time
- Limit breaches logged with structured event 'rate_limit_exceeded' for abuse pattern detection

13.8 AI Output Versioning

AI prompts, models, and post-processing logic will evolve over time. Without versioning, old AI outputs become indistinguishable from new ones, making it impossible to: (a) tell users their result is from an older model, (b) selectively re-run extractions when prompts improve, or (c) audit which model produced a given recommendation.

13.8.1 Schema Updates

```
-- climate_analysis
ALTER TABLE climate_analysis
  ADD COLUMN IF NOT EXISTS ai_version    TEXT NOT NULL DEFAULT 'v1.0',
  ADD COLUMN IF NOT EXISTS prompt_hash   TEXT,          -- SHA-256 of the prompt
template used
  ADD COLUMN IF NOT EXISTS llm_model     TEXT;          -- e.g. 'gemini-2.0-flash'

-- query_results – (column already partially present, formalised here)
ALTER TABLE query_results
  ADD COLUMN IF NOT EXISTS ai_version    TEXT NOT NULL DEFAULT 'v1.0',
  ADD COLUMN IF NOT EXISTS prompt_hash   TEXT;

-- New table – ai_versions registry
CREATE TABLE ai_versions (
  version      TEXT PRIMARY KEY,          -- e.g. 'v1.0', 'v1.1'
  description   TEXT NOT NULL,
```

```

prompt_template TEXT NOT NULL,
llm_model       TEXT NOT NULL,
effective_from  TIMESTAMPTZ NOT NULL DEFAULT now(),
retired_at      TIMESTAMPTZ          -- NULL while active
);

INSERT INTO ai_versions (version, description, prompt_template, llm_model)
VALUES ('v1.0', 'Initial extraction prompt – Gemini 2.0 Flash',
       '<full prompt template>', 'gemini-2.0-flash');

```

13.8.2 How It Is Used

- Every AI write records the active ai_version (queried from ai_versions where retired_at IS NULL)
- UI displays a small 'AI v1.0' tag on each result so users know which generation produced it
- Backfill job (admin-triggered): when a new ai_version ships, operators can selectively re-run extractions on documents with ai_version < new_version
- Result caching keys include ai_version — same query under v1.1 produces a fresh result, not a cached v1.0 answer

13.9 Climate Data Source — Clarification and Phase 2 Plan

With this Production Hardening Step explicitly clarifies the source of climate data fields (AQI, rainfall, temperature, location) to address ambiguity in earlier reviews.

13.9.1 MVP Source — NGO-Authored Documents (Authoritative)

In MVP, climate data is extracted from the NGO-uploaded documents themselves via the Gemini 2.0 Flash extraction step (Section 6.3, Step 12). This is by design — the platform's value proposition is peer NGO knowledge, so the climate context attached to each piece of knowledge comes from the NGO that wrote it. The data is what that NGO observed in their region.

Important — If an NGO document does not contain explicit climate values, the extraction LLM returns NULL for that field. The system NEVER fabricates climate data. NULL fields simply do not contribute to the Climate Twin Matching layer (Step C.3, Layer 2 — Climate Metadata) for that document.

13.9.2 Phase 2 — External Climate API Augmentation (Roadmap)

In Phase 2 (post-launch), the platform may augment NGO-provided data with external authoritative sources to fill gaps. This is documented here so the data model supports the eventual addition without rework.

Source	Provides	Status
OpenWeatherMap API	Real-time temperature, humidity, rainfall by lat/long	Phase 2 — investigate free tier limits
AQICN / WAQI	Real-time AQI by city	Phase 2 — has free tier

API		
IMD India / NOAA	Historical climate normals (decades)	Phase 2 — research-grade quality, low rate limits
Copernicus Climate	Climate model projections	Roadmap only — too heavy for MVP

13.9.3 Schema Provision for Phase 2

```
-- Add to climate_analysis (no behaviour change in MVP – fields just stay NULL)
ALTER TABLE climate_analysis
  ADD COLUMN IF NOT EXISTS source      TEXT NOT NULL DEFAULT 'NGO_DOCUMENT',
  ADD COLUMN IF NOT EXISTS source_api  TEXT,          -- 'openweather', 'aqicn',
etc. (Phase 2)
  ADD COLUMN IF NOT EXISTS fetched_at  TIMESTAMPTZ; -- when external data was
retrieved

-- Phase 2 – periodic enrichment cron job will:
-- 1. Find documents with NULL climate fields and a non-NULL location
-- 2. Geocode the location -> lat/long
-- 3. Call external API for that location
-- 4. INSERT a separate climate_analysis row with source = '<api>'
-- Original NGO-authored row is preserved.
```

13.10 Multi-Tenancy — Organisation-Level Data Isolation

Although the platform serves global NGOs, each NGO's data must be isolated such that one organisation cannot accidentally read another's draft documents, query history, or KYC details. With this Production Hardening Step codifies the isolation contract.

13.10.1 Tenant Boundary

Tenant = organisation_profile.id. Every user belongs to exactly one organisation; permissions are scoped at the organisation level, not the user level.

13.10.2 Schema Enforcement

Every business table that contains user-generated content carries a non-null org_id:

```
-- Already implicit via user_id, but with this Production Hardening Step makes it
explicit and indexed
ALTER TABLE documents
  ADD COLUMN IF NOT EXISTS org_id UUID REFERENCES organisation_profiles(id);

UPDATE documents d
  SET org_id = up.organisation_id
  FROM user_profiles up
 WHERE up.user_id = d.user_id
```

```

    AND d.org_id IS NULL;

ALTER TABLE documents ALTER COLUMN org_id SET NOT NULL;
CREATE INDEX idx_documents_org ON documents(org_id, created_at DESC);

-- Same pattern for: query_jobs, query_results, climate_analysis, embeddings,
insights, solutions

```

13.10.3 Row-Level Security via Application Layer

The platform enforces tenancy at the application layer (not Postgres RLS) for transparency and performance. Every data-access function in the Go and Python services takes an `org_id` parameter, and SQL queries always include `'WHERE org_id = $org_id'` as the first predicate.

```

// Go – repository pattern
func (r *DocumentRepo) GetByID(ctx context.Context, orgID, docID uuid.UUID)
(*Document, error) {
    var doc Document
    err := r.db.QueryRowContext(ctx, `
        SELECT id, user_id, file_name, status, ...
        FROM documents
        WHERE id = $1 AND org_id = $2`,
        docID, orgID).Scan(&doc.ID, &doc.UserID, ...)
    return &doc, err
}

// orgID is extracted from the JWT claim "org" by middleware – never from request
body

```

13.10.4 Cross-Org Visibility (The Climate Twin Exception)

Climate Twin Matching deliberately reaches across orgs — that is the entire product. The exception is contained: only DOCUMENTS marked status = COMPLETED and visibility = PUBLIC are included in vector search results across orgs. KYC documents, draft uploads, and private query history are never visible cross-org.

```

ALTER TABLE documents
    ADD COLUMN IF NOT EXISTS visibility TEXT NOT NULL DEFAULT 'PUBLIC'
    CHECK (visibility IN ('PUBLIC', 'ORG_ONLY', 'PRIVATE'));

-- Cross-org vector search filter (Section 7.5 B.4 – updated query)
SELECT d.id, d.org_id, e.embedding <=> $query_embedding AS distance
FROM embeddings e
JOIN documents d ON d.id = e.doc_id
WHERE d.status = 'COMPLETED'
    AND d.visibility = 'PUBLIC'          -- NEW: only public docs are climate-twin
candidates
ORDER BY distance
LIMIT 50;

```

13.11 Summary — All Changes At a Glance

Area	Update	Section
Failure handling	retry_count, error_reason columns on documents and query_jobs	13.1
Status lifecycle	Added NEEDS_REVIEW state for low-confidence AI outputs	13.1.4
Logging	Mandatory structured JSON log schema for all services	13.2.1
Metrics	9 CloudWatch alerts + 4 custom metrics	13.2.2 / 13.2.3
AI reliability	Pydantic schema validation gate before persisting LLM output	13.3.1
AI reliability	Confidence threshold (< 0.75 -> NEEDS_REVIEW)	13.3.2
AI reliability	Sanity checks for AQI / rainfall / temperature ranges	13.3.3
AI reliability	Human review queue (Module 7) + ai_reviews audit table	13.3.4
Idempotency	idempotency_key on every SQS event + processed_events table	13.4
File upload	Pre-signed S3 URLs replace api-service-mediated upload	13.5
Real-time	Reaffirmed: SSE primary, polling fallback (already in v2.1)	13.6
Rate limiting	Per-user-per-day quotas for uploads (20) and queries (100)	13.7
AI versioning	ai_version column + ai_versions registry table	13.8
Climate data source	Documented: MVP from NGO docs, Phase 2 from external APIs	13.9
Multi-tenancy	Mandatory org_id on every business table, app-layer enforcement	13.10

Decisions Not Adopted — Caching layer (Redis) was evaluated and rejected at 2,000 users / ~3,000 daily queries, PostgreSQL with appropriate indexes handles all access patterns within latency targets. Re-evaluate if active user count exceeds 20,000 or query volume exceeds 50,000/day. Postgres tsvector + PGVector hybrid search is already implemented (Section 8.8).

14. Future Roadmap (Pie-in-Sky)

Several capabilities are planned beyond MVP. The architecture is designed so each can be added without rewriting existing modules.

14.1 Pie-in-Sky Features

Feature	Tech to add	Module(s) impacted
Mobile app (Android + iOS)	React Native (Expo) + WatermelonDB + MMKV	New mobile app calls existing API. FCM added to Module 6.
SMS OTP (alongside email)	AWS SNS	Module 6 (Notification) adds AWS SNS path
OAuth login (Google)	golang.org/x/oauth2	Module 1 (Auth) adds OAuth2 endpoints
MFA (TOTP)	RFC 6238 — github.com/pquerna/otp	Module 1 (Auth) adds /auth/mfa/* endpoints
Automated KYC	Onfido or Veriff API	Module 7 (Admin) integrates third-party KYC
Conversational chatbot	Claude Haiku 3.5 + RAG	Module 5 reuses existing PGVector + LLM stack
Donation management	Stripe Connect / Razorpay	New module: Donation Service
Multilingual content	react-i18next (frontend) + Postgres translation tables	Frontend + Module 2 (User) adds language preference
Migration to Pinecone	Pinecone managed vector DB	Module 5 swaps PGVector → Pinecone (only if > 5M embeddings)

14.2 Architectural Evolution

Trigger	Evolution
10K+ users with measured AI scaling pain	Extract Module 5 sub-components (parsing vs synthesis) into separate Fargate services
Team grows past ~15 engineers	Extract Module 7 (Admin) and Module 1 (Auth) into separate services
50K+ users	Selective microservice extraction based on measured hot-spots
Multi-region launch	Add CloudFront, deploy ai-service in multiple regions

15. Final Word

This Final Low-Level Design Document captures the definitive architecture, technology stack, and implementation specification for the Climate Intelligence Exchange platform.

The platform is built as a modular monolith deployed as two services on AWS ECS Fargate. Eight logical modules — Auth, User, Document, Query, AI Intelligence, Notification, Admin, and Search — are organised with clean internal boundaries, deployed cost-effectively, and ready to be extracted into separate services when measured pain demands it.

Through systematic evaluation of every component against the real operating constraints — approximately 2,000 global NGO users, a small team, a cost-first optimization priority, and the critical responsibility to serve low-bandwidth users like the Shweta persona — the stack was deliberately simplified. Eleven technologies were removed from earlier iterations and fifteen substitutions were made: not to cut corners, but to eliminate complexity that this scale does not justify.

This stack delivers a production-grade, globally-deployable platform that meets every Grow+ requirement — from the Johnson persona's need for credible climate intelligence to the Shweta persona's requirement for low-bandwidth accessibility. It scales smoothly from 2,000 to 50,000 users without architectural rewrites. It runs on two deployable services instead of eight. It costs approximately \$230 to \$280 per month in total infrastructure. And every choice was made deliberately — where a simpler or cheaper alternative existed without capability loss it was adopted.

This is the recommended path forward for implementation.

Ready for Engineering Handoff:

Any changes from this point forward must be tracked through formal version control and approved by the platform architecture lead.